

Capstone: Ovarian Cancer Prediction Simulation

HI 192: Knowledge Representation and Health Decision Support

DELA CRUZ James Angelo **DORIA**, Marc **RODRIGUEZ**,
Augustus Clark Raphael

May 2026

Contents

1	Introduction	7
2	Dataset	8
2.1	Data Source	8
2.2	Class Distribution	8
2.3	Splitting the Data	8
2.4	Data Integrity	9
3	Preprocessing	10
3.1	Raw Data Inventory	10
3.2	Normalization	10
3.3	Data Augmentation	10
3.4	Ensuring Experimental Integrity	11
4	Experimental Pipeline	12
5	Methodology	13
5.1	Experiment Setup	13
5.2	Architecture and Model Design	13
5.3	Hyperparameters and Optimization	14
5.4	Metrics and Evaluation	14
6	Results	15
6.1	General Performance	15
6.2	Top Performers	15
6.3	The Impact of Learning Rate	15
6.4	Optimizer Comparison	15
6.5	Visualizing the Results	16
6.6	Failures and Suboptimal Runs	16
7	Discussion	17
7.1	Why Learning Rate Controlled Everything	17
7.2	Comparing the Optimizers	17
7.3	Architecture Strengths	18
7.4	A Warning about AUC	18
7.5	Recommendations	18
8	Conclusion	19

9	Appendices	20
9.1	Per-Model Accuracy and Loss Curves	20
9.2	Per-Model AUC-ROC Curves	29
9.3	Per-Model Confusion Matrices	37
9.4	Complete 72-Run Results Table	45
9.5	Source Code	47

List of Figures

1	Experiment Pipeline	12
2	Full Accuracy Heatmap	16
3	Full AUC-ROC Heatmap	17
4	VGG19,Adam: Accuracy and Loss Curves across Learning Rates . . .	21
5	VGG19,Adagrad: Accuracy and Loss Curves across Learning Rates .	21
6	VGG19,Adamax: Accuracy and Loss Curves across Learning Rates .	21
7	VGG19,AdaDelta: Accuracy and Loss Curves across Learning Rates .	22
8	VGG19,SGD: Accuracy and Loss Curves across Learning Rates . . .	22
9	VGG19,RMSProp: Accuracy and Loss Curves across Learning Rates .	22
10	EfficientNetB3,Adam: Accuracy and Loss Curves across Learning Rates	23
11	EfficientNetB3,Adagrad: Accuracy and Loss Curves across Learning Rates	23
12	EfficientNetB3,Adamax: Accuracy and Loss Curves across Learning Rates	23
13	EfficientNetB3,AdaDelta: Accuracy and Loss Curves across Learning Rates	24
14	EfficientNetB3,SGD: Accuracy and Loss Curves across Learning Rates	24
15	EfficientNetB3,RMSProp: Accuracy and Loss Curves across Learning Rates	24
16	ResNet50,Adam: Accuracy and Loss Curves across Learning Rates . .	25
17	ResNet50,Adagrad: Accuracy and Loss Curves across Learning Rates	25
18	ResNet50,Adamax: Accuracy and Loss Curves across Learning Rates	25
19	ResNet50,AdaDelta: Accuracy and Loss Curves across Learning Rates	26
20	ResNet50,SGD: Accuracy and Loss Curves across Learning Rates . .	26
21	ResNet50,RMSProp: Accuracy and Loss Curves across Learning Rates	26
22	DenseNet121,Adam: Accuracy and Loss Curves across Learning Rates	27
23	DenseNet121,Adagrad: Accuracy and Loss Curves across Learning Rates	27
24	DenseNet121,Adamax: Accuracy and Loss Curves across Learning Rates	27
25	DenseNet121,AdaDelta: Accuracy and Loss Curves across Learning Rates	28
26	DenseNet121,SGD: Accuracy and Loss Curves across Learning Rates	28
27	DenseNet121,RMSProp: Accuracy and Loss Curves across Learning Rates	28
28	VGG19,Adam: AUC-ROC Curves across Learning Rates	29
29	VGG19,Adagrad: AUC-ROC Curves across Learning Rates	29
30	VGG19,Adamax: AUC-ROC Curves across Learning Rates	29
31	VGG19,AdaDelta: AUC-ROC Curves across Learning Rates	30
32	VGG19,SGD: AUC-ROC Curves across Learning Rates	30
33	VGG19,RMSProp: AUC-ROC Curves across Learning Rates	30
34	EfficientNetB3,Adam: AUC-ROC Curves across Learning Rates . . .	31

35	EfficientNetB3,Adagrad: AUC-ROC Curves across Learning Rates . .	31
36	EfficientNetB3,Adamax: AUC-ROC Curves across Learning Rates . .	31
37	EfficientNetB3,AdaDelta: AUC-ROC Curves across Learning Rates .	32
38	EfficientNetB3,SGD: AUC-ROC Curves across Learning Rates	32
39	EfficientNetB3,RMSProp: AUC-ROC Curves across Learning Rates .	32
40	ResNet50,Adam: AUC-ROC Curves across Learning Rates	33
41	ResNet50,Adagrad: AUC-ROC Curves across Learning Rates	33
42	ResNet50,Adamax: AUC-ROC Curves across Learning Rates	33
43	ResNet50,AdaDelta: AUC-ROC Curves across Learning Rates	34
44	ResNet50,SGD: AUC-ROC Curves across Learning Rates	34
45	ResNet50,RMSProp: AUC-ROC Curves across Learning Rates	34
46	DenseNet121,Adam: AUC-ROC Curves across Learning Rates	35
47	DenseNet121,Adagrad: AUC-ROC Curves across Learning Rates . . .	35
48	DenseNet121,Adamax: AUC-ROC Curves across Learning Rates . . .	35
49	DenseNet121,AdaDelta: AUC-ROC Curves across Learning Rates . .	36
50	DenseNet121,SGD: AUC-ROC Curves across Learning Rates	36
51	DenseNet121,RMSProp: AUC-ROC Curves across Learning Rates . .	36
52	VGG19,Adam: Confusion Matrices across Learning Rates	37
53	VGG19,Adagrad: Confusion Matrices across Learning Rates	37
54	VGG19,Adamax: Confusion Matrices across Learning Rates	37
55	VGG19,AdaDelta: Confusion Matrices across Learning Rates	38
56	VGG19,SGD: Confusion Matrices across Learning Rates	38
57	VGG19,RMSProp: Confusion Matrices across Learning Rates	38
58	EfficientNetB3,Adam: Confusion Matrices across Learning Rates . . .	39
59	EfficientNetB3,Adagrad: Confusion Matrices across Learning Rates .	39
60	EfficientNetB3,Adamax: Confusion Matrices across Learning Rates .	39
61	EfficientNetB3,AdaDelta: Confusion Matrices across Learning Rates .	40
62	EfficientNetB3,SGD: Confusion Matrices across Learning Rates . . .	40
63	EfficientNetB3,RMSProp: Confusion Matrices across Learning Rates	40
64	ResNet50,Adam: Confusion Matrices across Learning Rates	41
65	ResNet50,Adagrad: Confusion Matrices across Learning Rates	41
66	ResNet50,Adamax: Confusion Matrices across Learning Rates	41
67	ResNet50,AdaDelta: Confusion Matrices across Learning Rates	42
68	ResNet50,SGD: Confusion Matrices across Learning Rates	42
69	ResNet50,RMSProp: Confusion Matrices across Learning Rates . . .	42
70	DenseNet121,Adam: Confusion Matrices across Learning Rates	43
71	DenseNet121,Adagrad: Confusion Matrices across Learning Rates . .	43
72	DenseNet121,Adamax: Confusion Matrices across Learning Rates . .	43
73	DenseNet121,AdaDelta: Confusion Matrices across Learning Rates . .	44

74	DenseNet121,SGD: Confusion Matrices across Learning Rates	44
75	DenseNet121,RMSProp: Confusion Matrices across Learning Rates .	44

List of Tables

1	Breakdown of images by class in the STRAMPN dataset.	8
2	Stratified train, validation, and test split.	9
3	Augmentation settings applied to the training set.	11
4	Classification Head	13
5	Experiment Matrix	14
6	Per-Architecture AUC Ranking	15
7	Complete 72-run results table sorted by AUC-ROC descending. . . .	46

1 Introduction

Ovarian cancer is one of the most serious challenges in gynecological oncology, often diagnosed at late stages where the prognosis is poor. Early and accurate detection is critical, and histopathology remains the gold standard for diagnosis. However, manually analyzing these slides is time-consuming and depends heavily on the expert's experience. With the rise of deep learning, there is a lot of interest in using computer vision to help automate this process, especially with transfer learning which allows us to use powerful models pre-trained on massive datasets like ImageNet even when we have limited medical data.

In this project, we wanted to see how different optimization settings affect the performance of these models when applied to histopathological images. Instead of just picking one model and one optimizer, we conducted a systematic study of 72 different configurations. We combined four well-established architectures, namely VGG19, EfficientNetB3, ResNet50, and DenseNet121, with six different optimizers and three learning rates. The goal was to find which settings are the most reliable for this specific task and to understand why some configurations fail while others succeed.

This paper is organized as follows: we first describe the STRAMPN dataset and our preprocessing steps. Then, we outline our experimental setup and the specific architectures and optimizers we tested. Finally, we present our results, discuss the trade-offs we observed between different settings, and conclude with recommendations for future research in automated cancer classification.

2 Dataset

2.1 Data Source

For this study, we used the **STRAMPN Histopathological Images for Ovarian Cancer Prediction** dataset from IEEE DataPort. It can be found at:

[https://ieee-dataport.org/documents/
strampn-histopathological-images-ovarian-cancer-prediction](https://ieee-dataport.org/documents/strampn-histopathological-images-ovarian-cancer-prediction)

The dataset contains histopathological JPEG images of ovarian tissue, labeled as either *Ovarian Cancer* or *Ovarian Non-Cancer*.

2.2 Class Distribution

There are 987 images in total. As shown in Table 1, the dataset is reasonably balanced.

Table 1: Breakdown of images by class in the STRAMPN dataset.

Class	Count	Percentage
Ovarian Cancer	481	48.73%
Ovarian Non-Cancer	506	51.27%
Total	987	100%

Since the split between cancer and non-cancer images is nearly 50/50 there is little concern in regards to class imbalance. Allowing accuracy and AUC to remain the primary metrics of focus in the simulation, with the a suite of metrics collected alongside the primary two (accuracy, recall, specificity, F1-score)

2.3 Splitting the Data

As specified, the dataset was split into training, validation, and test sets using a two-stage process. First, we took 75% of the images from each class for our training and validation pool, leaving 25% for the final test set. Then, we split that 75% pool again, using 70% for actual training and 30% for validation.

We used `sklearn's train_test_split` with `stratify=labels` to make sure the class proportions stayed the same in every subset. We also fixed the `random_state` to 42 so that our splits are completely reproducible. The final counts for each split are in Table 2.

Table 2: Stratified train, validation, and test split.

Split	Cancer	Non-Cancer	Total	% of Dataset
Train	253	265	518	52.48%
Validation	108	114	222	22.49%
Test	120	127	247	25.03%
Total	481	506	987	100%

By keeping the seed at 42, we ensured that every one of our 72 training runs used the exact same images for training and testing.

2.4 Data Integrity

After copying the files into their respective folders, we ran a quick check in our preprocessing script to make sure no images were accidentally placed in more than one split (data leakage). Our script confirmed that there were no overlaps:

```
1 PREPROCESSING COMPLETE: No integrity violations found
```

This gives us confidence that our evaluation results are based on truly unseen data.

3 Preprocessing

We handled all our preprocessing using a custom script, `src/preprocessing.py`. We used standard libraries like `pathlib` and `sklearn`, keeping this step separate from the training loop to ensure the data preparation was clear and repeatable.

3.1 Raw Data Inventory

First, we took an inventory of the raw dataset to make sure all files were valid JPEGs and that we had the expected counts for each class. The script confirmed that we had 481 cancer images and 506 non-cancer images, all in the correct format.

```

1 -----
2 STEP 1: RAW DATASET INVENTORY
3 -----
4 [cancer]      Total files: 481 [ok]
5 [no_cancer]   Total files: 506 [ok]
6 Total images: 987 [ok]

```

3.2 Normalization

We applied **samplewise standard normalization** to every image using Keras's `ImageDataGenerator`. This involved:

- Centering each image by subtracting its mean pixel value.
- Standardizing each image by dividing by its standard deviation.

Samplewise normalization was chosen for its reliability for relatively small datasets such as this one. As opposed to relying on global statistics (which can be noisy or lead to data leakage), each image is processed independently. This makes the evaluation more reliable and ensures the model handles new images correctly during testing.

3.3 Data Augmentation

To help the models learn more robust features and avoid overfitting, we used data augmentation, but **only for the training set**. We kept the validation and test sets in their original state (except for normalization) to ensure the results reflect real-world performance.

Table 3 summarizes the augmentations we used. We picked these to simulate the typical variations found in histology slides, such as different rotations and slight scaling differences.

Table 3: Augmentation settings applied to the training set.

Setting	Value	Goal
rotation_range	30°	Handle slides at any angle
width/height shift	0.15	Account for slight centering variations
shear_range	0.3	Simulate tissue distortion
zoom_range	0.30	Handle magnification differences
horizontal_flip	True	Tissue features are orientation-independent

3.4 Ensuring Experimental Integrity

A key part of our methodology was ensuring that the data generators were only initialized **once** at the start of the entire 72-run simulation. If we had recreated the generators inside the training loop, each model would have seen the training images in a different order, adding "luck of the draw" as a hidden factor in our results. By sharing the same generators, we ensured that every architecture and optimizer was tested on the exact same sequence of data.

4 Experimental Pipeline

The pipeline was designed to be as consistent as possible across all 72 configurations. We wanted to make sure that the only things changing were the architecture, optimizer, and learning rate all according to specification. The process follows a clear path from raw data to final evaluation.

The diagram in Figure 1 shows the high-level flow. Starting by preparing the data (splitting and augmenting), then move into the core training loop where we iterate through our experimental matrix, and finally aggregate all the metrics for analysis.

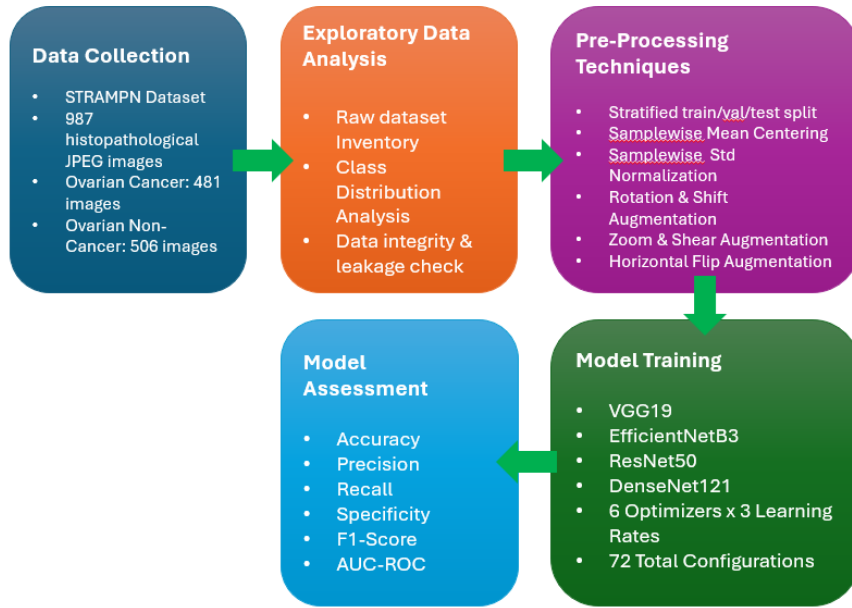


Figure 1: Our experimental workflow. We used a unified data pipeline for all 72 runs to ensure that any differences in performance were due to the model settings, not the data preparation.

By using this structured approach, we were able to systematically compare how each architecture handled different optimizers. This setup also made it easier to identify suboptimal runs early, as we could see when a specific combination of learning rate and optimizer failed to converge regardless of the base model.

5 Methodology

5.1 Experiment Setup

The simulation was built around a full factorial design to see how different combinations of models and optimizers perform. We tested 4 base architectures, 6 optimizers, and 3 learning rates, which gave us a total of $4 \times 6 \times 3 = \mathbf{72}$ unique runs. Each run followed the same training process on our fixed data split to ensure a fair comparison.

We automated the entire process using a master script (`src/run_all.py`) that iterated through every configuration. To keep things organized, each run was saved in its own folder named after the architecture, optimizer, and learning rate (e.g., `VGG19_Adam_LR1e-04/`).

5.2 Architecture and Model Design

We used four convolutional neural networks as our base models: VGG19, EfficientNetB3, ResNet50, and DenseNet121. Following standard transfer learning practices, we loaded these with ImageNet weights and froze all their convolutional layers. This allowed us to use the powerful features these models already learned from millions of images, while we only trained a new classification head on top.

The design of the classification head (Table 4) was provided in the project specifications. It uses a series of Dense layers with Batch Normalization and Dropout to prevent overfitting on our relatively small dataset. We used GlobalAveragePooling2D to bridge the gap between the frozen base and our dense layers, a pattern we adapted from our course materials.

Table 4: Our standard classification head (used for all 72 runs).

Layer	Parameters	Activation
GlobalAveragePooling2D	—	—
Dense	256 units	ReLU
BatchNormalization	—	—
Dropout	0.6 rate	—
Dense	128 units	ReLU
BatchNormalization	—	—
Dropout	0.4 rate	—
Dense	64 units	ReLU
BatchNormalization	—	—
Dropout	0.3 rate	—
Dense (Output)	1 unit	Sigmoid

5.3 Hyperparameters and Optimization

The simulation tested six different optimizers: Adam, Adagrad, Adamax, AdaDelta, SGD (with 0.9 momentum), and RMSProp. For each optimizer, we tried three learning rates: 10^{-4} , 10^{-5} , and 10^{-6} .

Our training parameters were also fixed across all runs:

- **Input Resolution:** 256×256
- **Batch Size:** 32
- **Maximum Epochs:** 50
- **Loss Function:** Binary Crossentropy

To make training efficient, we used several callbacks. We set up `EarlyStopping` to halt runs that stopped improving after 10 epochs, `ReduceLROnPlateau` to adjust the learning rate if the validation loss stalled, and `ModelCheckpoint` to keep only the best weights from each run.

5.4 Metrics and Evaluation

After each run, we evaluated the model on our held-out test set (247 images). We collected the specified metrics: accuracy, precision, and recall, with AUC-ROC as our primary way to rank the models.

Since specificity isn't a default metric in Keras, we implemented a custom calculation to extract it from the confusion matrix. This was important because, in medical imaging, knowing how well a model identifies healthy (non-cancer) tissue is just as critical as finding the cancer.

Table 5: The full 72-run experiment matrix.

Run	Architecture	Optimizer / LR
1–18	VGG19	6 Optimizers $\times$ 3 LRs
19–36	EfficientNetB3	6 Optimizers $\times$ 3 LRs
37–54	ResNet50	6 Optimizers $\times$ 3 LRs
55–72	DenseNet121	6 Optimizers $\times$ 3 LRs

6 Results

6.1 General Performance

Overall, the 72 runs showed a wide range of results. We observed everything from near-perfect classification to complete model failure. The best performance came from VGG19 using Adam and Adamax at a learning rate of 10^{-4} , both reaching an AUC of 0.9995. On the other end of the spectrum, we had 10 suboptimal runs where the model failed to learn anything useful and just predicted one class for every image.

6.2 Top Performers

When we look at the best model for each architecture (Table 6), we see that every base model is capable of high performance if given the right optimizer. VGG19 and DenseNet121 both hit very high AUC scores with Adam, while ResNet50 and EfficientNetB3 performed best with RMSProp.

Table 6: Top configurations for each architecture (ranked by AUC).

Model	Optimizer	LR	Accuracy	AUC
VGG19	Adam	$1e-4$	77.33%	0.9995
DenseNet121	Adam	$1e-4$	96.76%	0.9991
ResNet50	RMSProp	$1e-4$	95.14%	0.9943
EfficientNetB3	RMSProp	$1e-4$	84.62%	0.9841

6.3 The Impact of Learning Rate

One of our biggest findings was just how much the learning rate matters for transfer learning. As we dropped the learning rate from 10^{-4} down to 10^{-6} , performance took a massive hit across the board.

At 10^{-4} , most models converged quickly and performed well. However, at 10^{-6} , many models stopped improving after just 11 epochs (due to our EarlyStopping callback) because the updates were too small to make any real progress. This tells us that for these specific frozen models, 10^{-4} is likely the optimal rate.

6.4 Optimizer Comparison

In our tests, RMSProp was the most consistent performer, with the highest average AUC across all models. Adam and Adamax were also very strong, especially at the

10^{-4} learning rate. On the other hand, AdaDelta and SGD struggled significantly on this dataset, especially when combined with smaller learning rates.

6.5 Visualizing the Results

We used heatmaps to visualize how the different architectures responded to the optimizer and learning rate settings. Figures 2 and 3 show that while some models are more flexible, most configurations cluster around a specific ideal zone in the hyperparameter space.

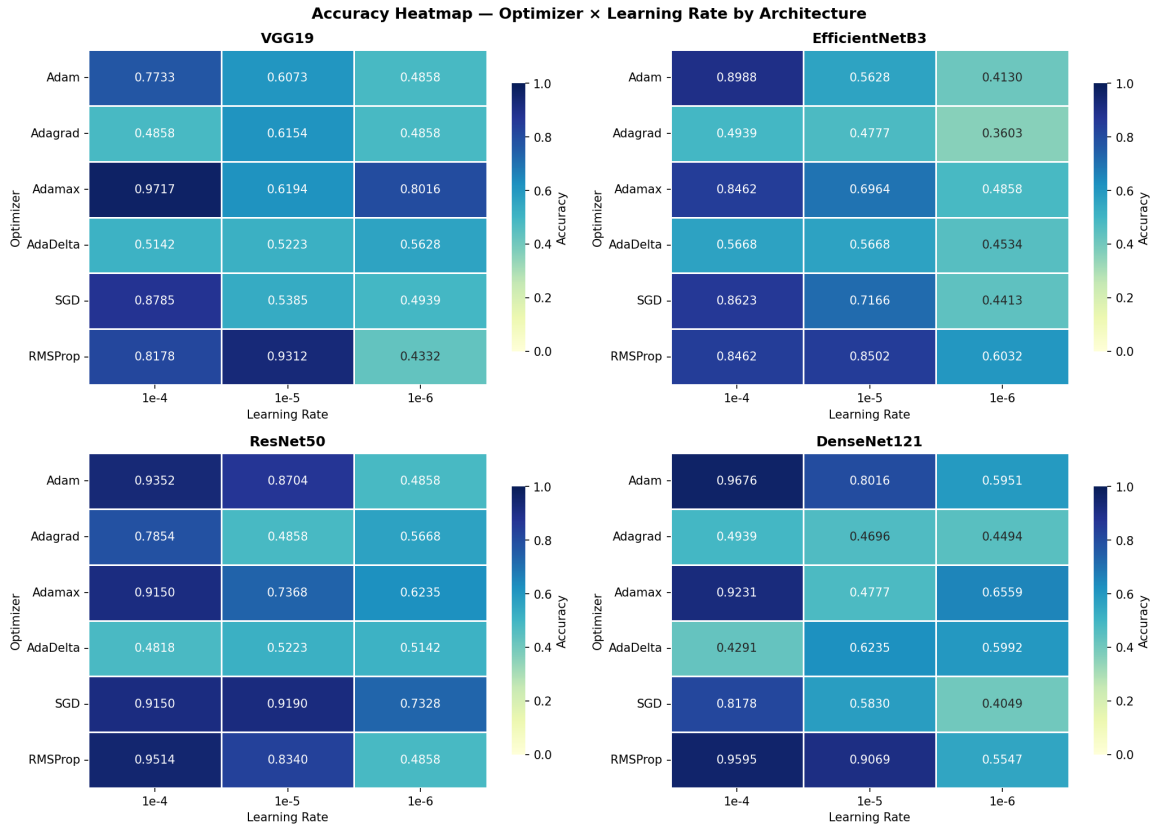


Figure 2: Accuracy across all 72 configurations.

6.6 Failures and Suboptimal Runs

It is also worth noting the runs that failed. We identified 10 cases where the models simply collapsed to predicting one class. Interestingly, some of these runs still showed a high AUC (above 0.80) even though they were practically useless because they never correctly identified a single cancer (or non-cancer) case. This highlighted why we need to look at more than just AUC when evaluating these models for medical use.

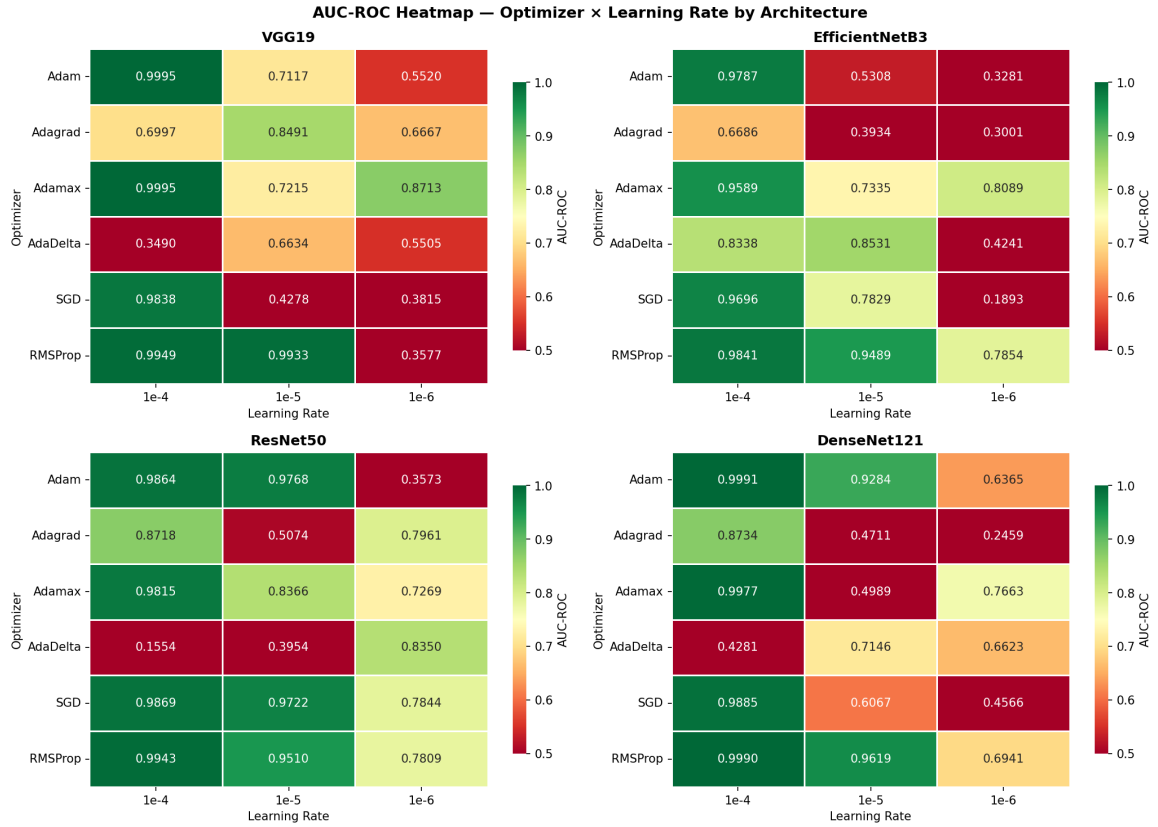


Figure 3: AUC-ROC across all 72 configurations.

7 Discussion

7.1 Why Learning Rate Controlled Everything

The results show a very clear pattern: the learning rate was the single most important factor in whether a model succeeded or failed. At 10^{-6} , almost every model struggled. This makes sense when you consider that we kept the base convolutional layers frozen. Since we were starting the classification head from scratch, those weights needed a decent-sized push to start learning the features extracted by the pre-trained model. At 10^{-6} , the updates were so tiny that the model barely moved from its initial random state before our `EarlyStopping` callback cut it off.

7.2 Comparing the Optimizers

A notable observation was how the different optimizers handled the relatively small dataset. RMSProp was the winner here, followed closely by Adam and Adamax. These adaptive methods are great for medical imaging projects like this because they can adjust their updates based on the gradients they see, which helps when the data is limited and potentially noisy.

In contrast, AdaDelta was surprisingly poor. It is meant to be an improvement

over Adagrad, but on the provided dataset, it seemed to stall almost immediately. This may be due to, with so few steps per epoch, AdaDelta’s internal learning rate calculation dropping to zero before the model could find a good path.

7.3 Architecture Strengths

ResNet50 was the most reliable performer across the architecture suite. This may be due to its skip connections helping keep the gradient signal clean as it passes back to the classification head. VGG19 actually rendered the single best individual run, which might be because its simpler, older architecture produces very straightforward features that the pooling-and-dense head could easily interpret.

EfficientNetB3, which is the most modern model tested, actually came in last on average. Which may be surprising, but it’s possible that its more complex feature space needs a lot more data than our 518 training images to really perform to specification.

7.4 A Warning about AUC

One of the most important takeaways for us was about the AUC metric itself. We saw several suboptimal runs where the model predicted the same class for every single image, yet they still had an AUC over 0.80. For instance, the VGG19 + Adam (LR= 10^{-6}) (Figure 52c) with gradient updates so miniscule, the model weights never meaningfully left from start; leading to all 247 test images to be predicted as cancer without exception, an F1 of 0, and a test accuracy of 48.58% (seemingly random chance) despite a retention of AUC 0.5520. This happens because AUC measures how well the model ranks images, not how well it classifies them at a specific threshold. In a real clinical setting, a model that never finds a single cancer case is a total failure, even if its ranking ability looks good on paper. This reinforces the need to always check recall and specificity alongside AUC.

7.5 Recommendations

Should the research be continued, the next logical step would be fine-tuning. We kept the base models frozen to see what a clean transfer learning performance looked like, but unfreezing the last few layers and training them at a very low learning rate would likely boost the scores significantly. It would be prudent to try k -fold cross-validation to get a better sense of how these models generalize across different subsets of the data.

8 Conclusion

In this study, we systematically tested 72 different deep learning configurations to see how they performed at classifying ovarian cancer images. By looking at four different architectures across six optimizers and three learning rates, we have gained a much clearer picture of what works for this kind of medical imaging task.

Our main findings are quite clear:

- **Learning Rate is Key:** For frozen transfer learning on this dataset, a learning rate of 10^{-4} is essential. Anything smaller often leads to model collapse.
- **Top Optimizers:** RMSProp, Adam, and Adamax are the most reliable choices for this task.
- **Reliable Models:** ResNet50 was the most consistent overall, while VGG19 produced the single best individual result.
- **Metric Caution:** We can't rely on AUC alone; checking for suboptimal runs using recall and specificity is critical for medical safety.

While these results are promising, they are just a starting point. We hope our findings provide a useful baseline for others working on similar medical classification problems, helping them choose the right optimization settings more quickly and avoid common pitfalls like suboptimal convergence.

9 Appendices

9.1 Per-Model Accuracy and Loss Curves

21

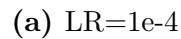


Figure 5: VGG19,Adagrad: Accuracy and Loss Curves across Learning Rates



Figure 5: VGG19,Adagrad: Accuracy and Loss Curves across Learning Rates

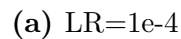
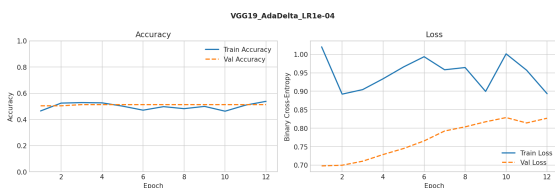
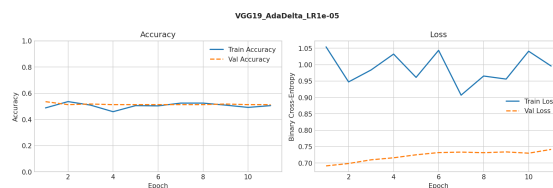


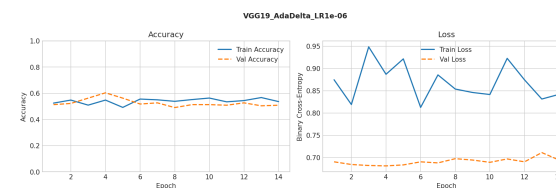
Figure 6: VGG19,Adamax: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

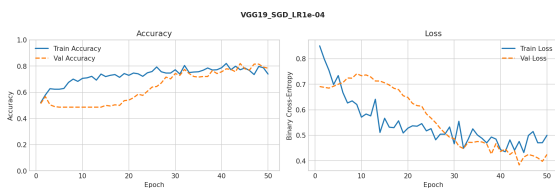


(b) LR=1e-5

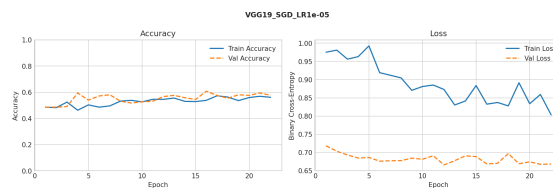


(c) LR=1e-6

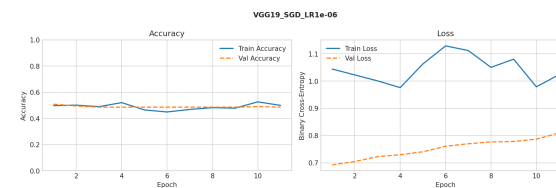
Figure 7: VGG19,AdaDelta: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

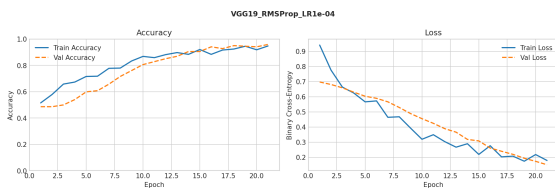


(b) LR=1e-5

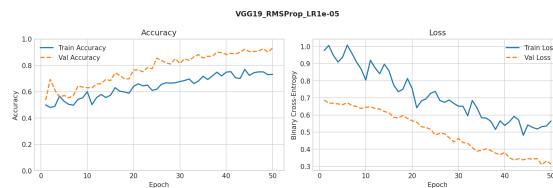


(c) LR=1e-6

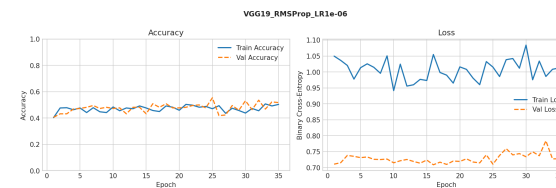
Figure 8: VGG19,SGD: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4



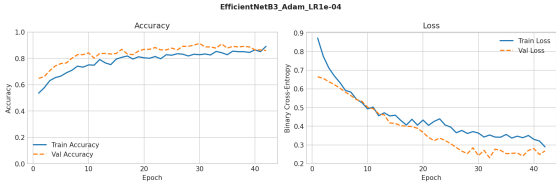
(b) LR=1e-5



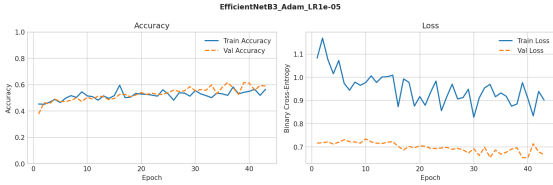
(c) LR=1e-6

Figure 9: VGG19,RMSProp: Accuracy and Loss Curves across Learning Rates

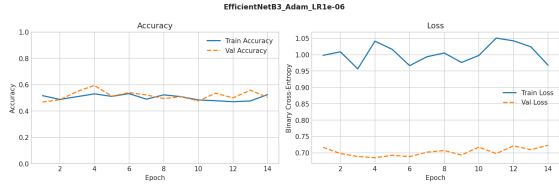
EfficientNetB3



(a) LR=1e-4

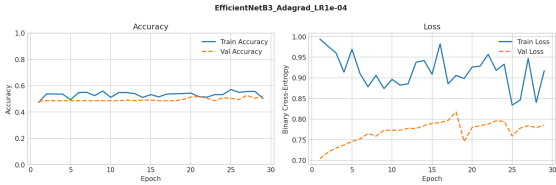


(b) LR=1e-5

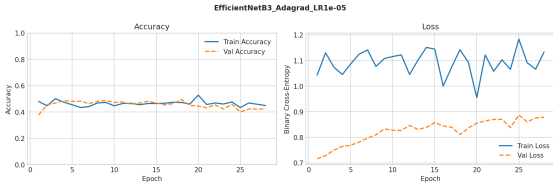


(c) LR=1e-6

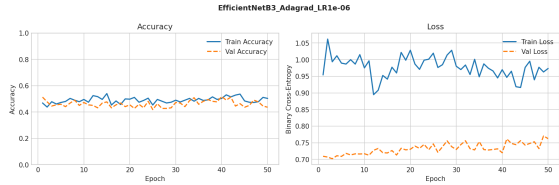
Figure 10: EfficientNetB3,Adam: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

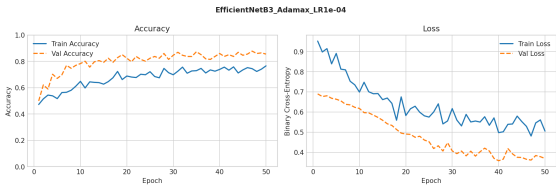


(b) LR=1e-5

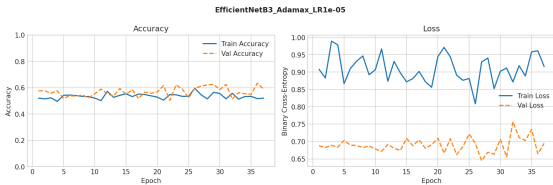


(c) LR=1e-6

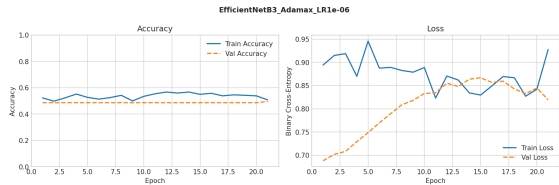
Figure 11: EfficientNetB3,Adagrad: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

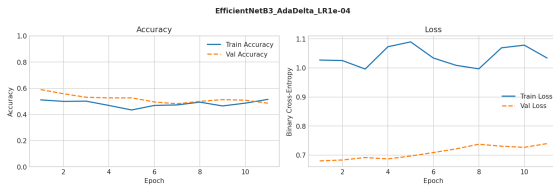


(b) LR=1e-5

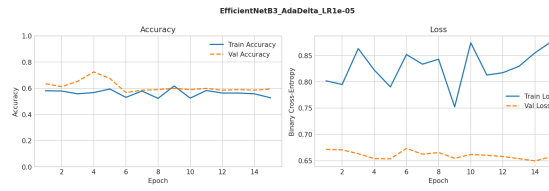


(c) LR=1e-6

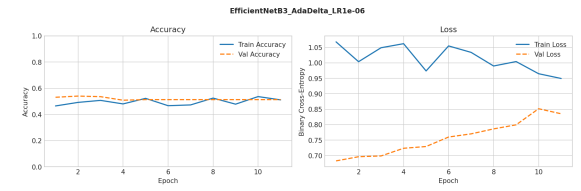
Figure 12: EfficientNetB3,Adamax: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

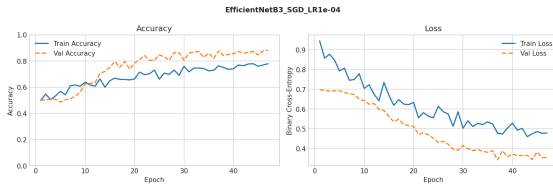


(b) LR=1e-5

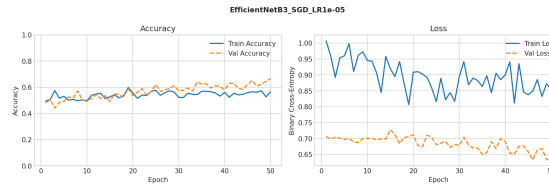


(c) LR=1e-6

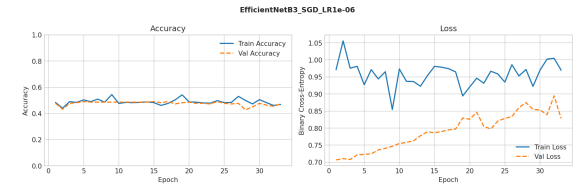
Figure 13: EfficientNetB3,AdaDelta: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

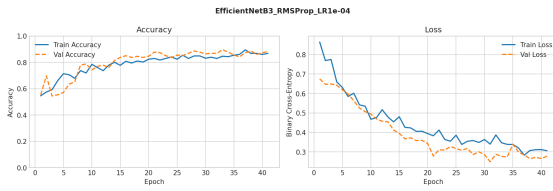


(b) LR=1e-5

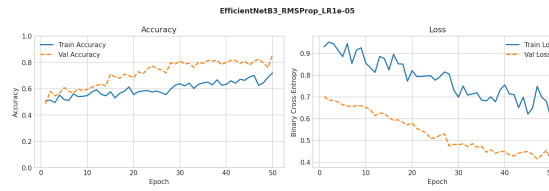


(c) LR=1e-6

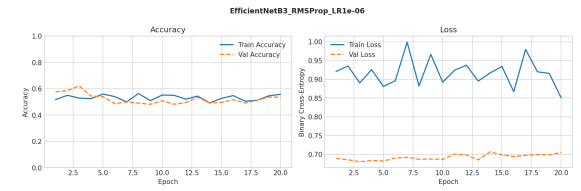
Figure 14: EfficientNetB3,SGD: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4



(b) LR=1e-5



(c) LR=1e-6

Figure 15: EfficientNetB3,RMSProp: Accuracy and Loss Curves across Learning Rates

ResNet50

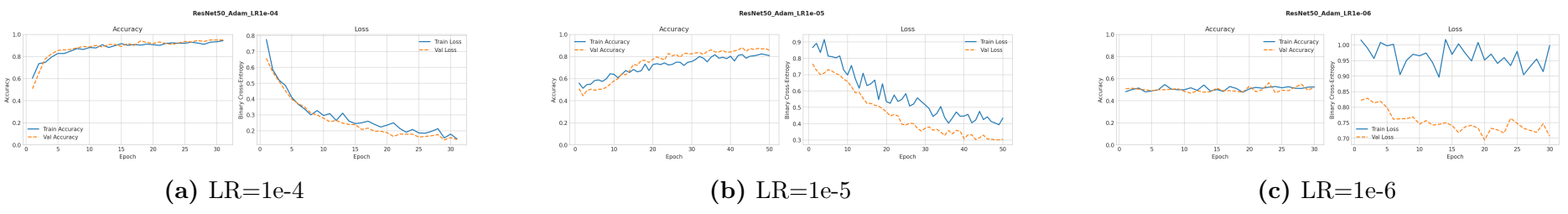


Figure 16: ResNet50,Adam: Accuracy and Loss Curves across Learning Rates

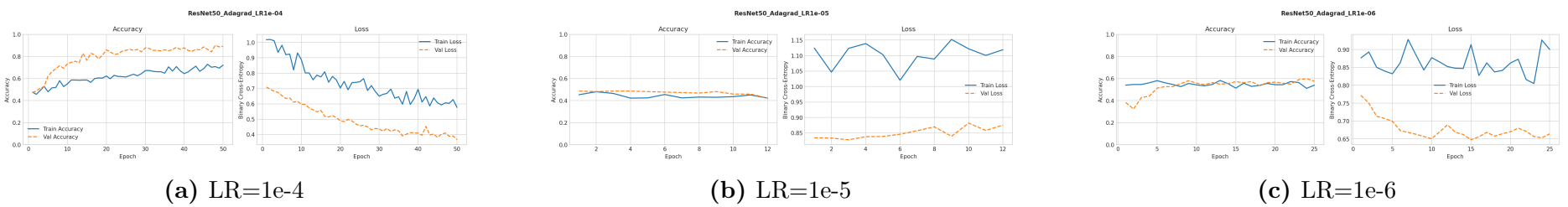


Figure 17: ResNet50,Adagrad: Accuracy and Loss Curves across Learning Rates

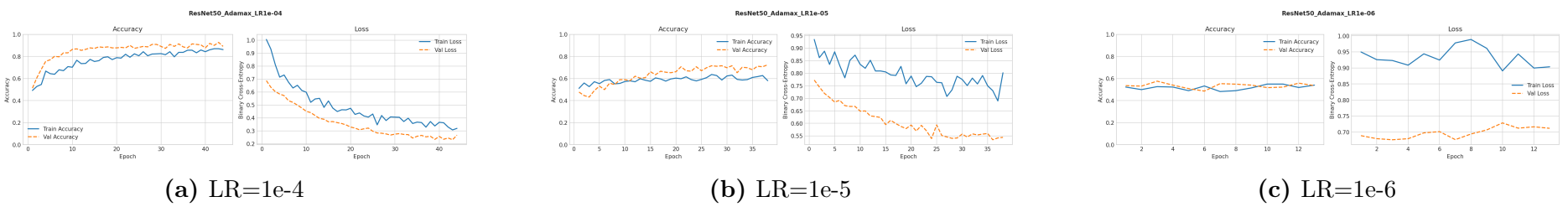


Figure 18: ResNet50,Adamax: Accuracy and Loss Curves across Learning Rates

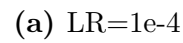


Figure 20: ResNet50,SGD: Accuracy and Loss Curves across Learning Rates

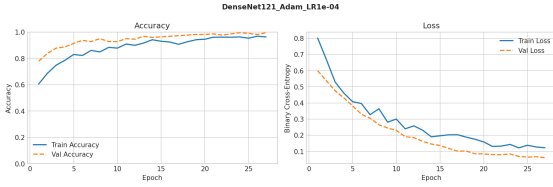


Figure 20: ResNet50,SGD: Accuracy and Loss Curves across Learning Rates

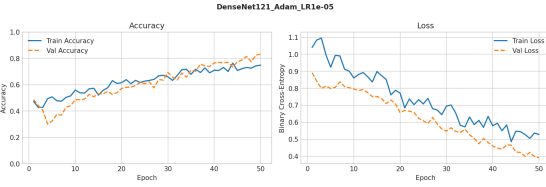


Figure 21: ResNet50,RMSProp: Accuracy and Loss Curves across Learning Rates

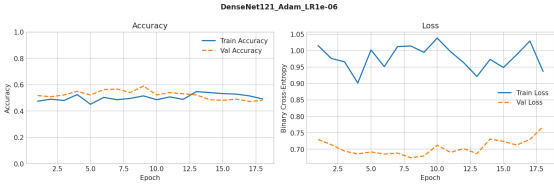
DenseNet121



(a) LR=1e-4

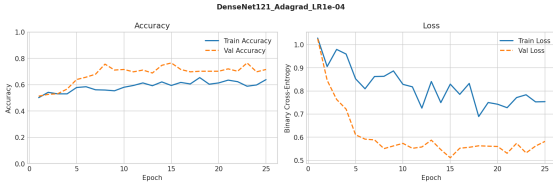


(b) LR=1e-5

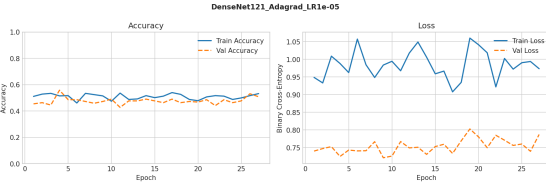


(c) LR=1e-6

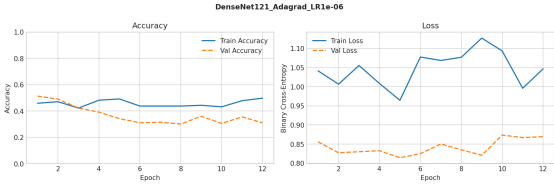
Figure 22: DenseNet121,Adam: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

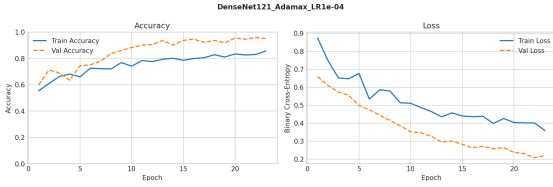


(b) LR=1e-5

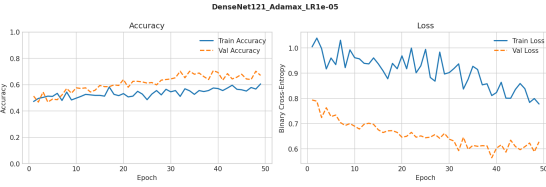


(c) LR=1e-6

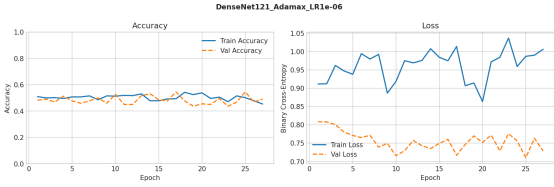
Figure 23: DenseNet121,Adagrad: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

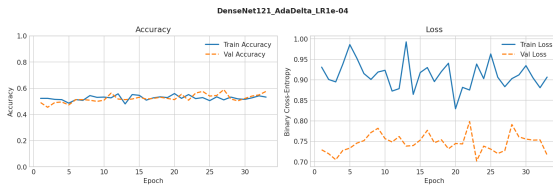


(b) LR=1e-5

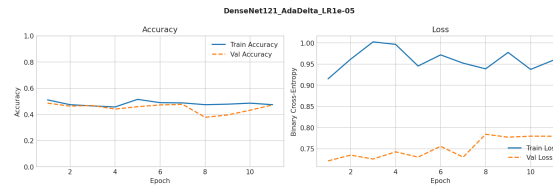


(c) LR=1e-6

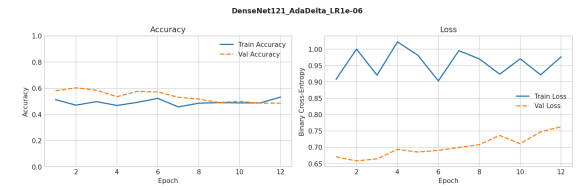
Figure 24: DenseNet121,Adamax: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

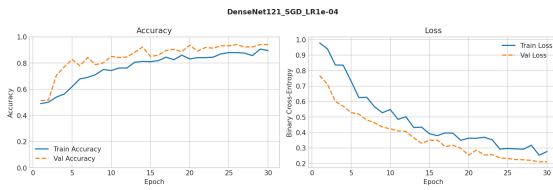


(b) LR=1e-5

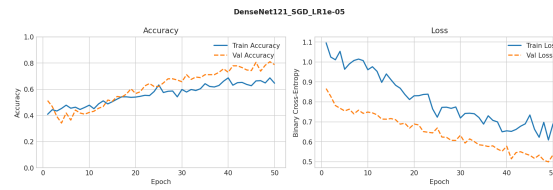


(c) LR=1e-6

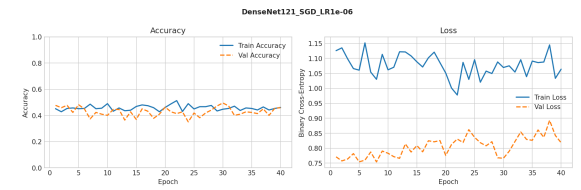
Figure 25: DenseNet121,AdaDelta: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

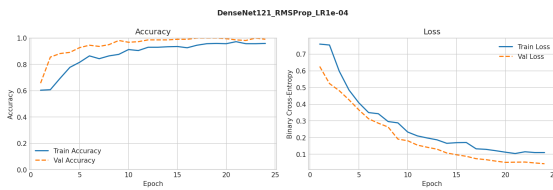


(b) LR=1e-5

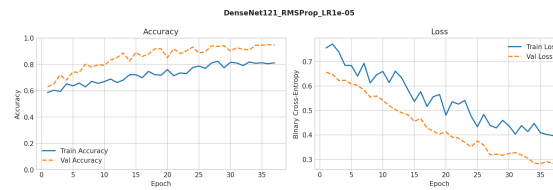


(c) LR=1e-6

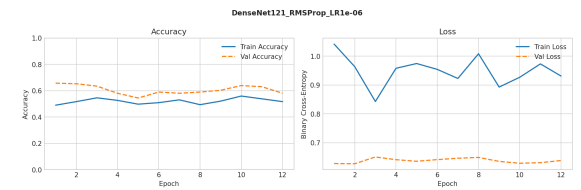
Figure 26: DenseNet121,SGD: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4



(b) LR=1e-5



(c) LR=1e-6

Figure 27: DenseNet121,RMSProp: Accuracy and Loss Curves across Learning Rates

9.2 Per-Model AUC-ROC Curves

VGG19

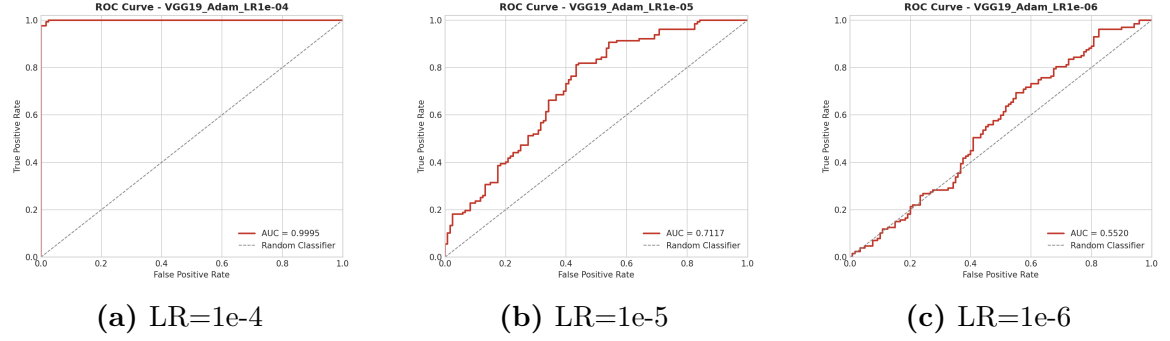


Figure 28: VGG19,Adam: AUC-ROC Curves across Learning Rates

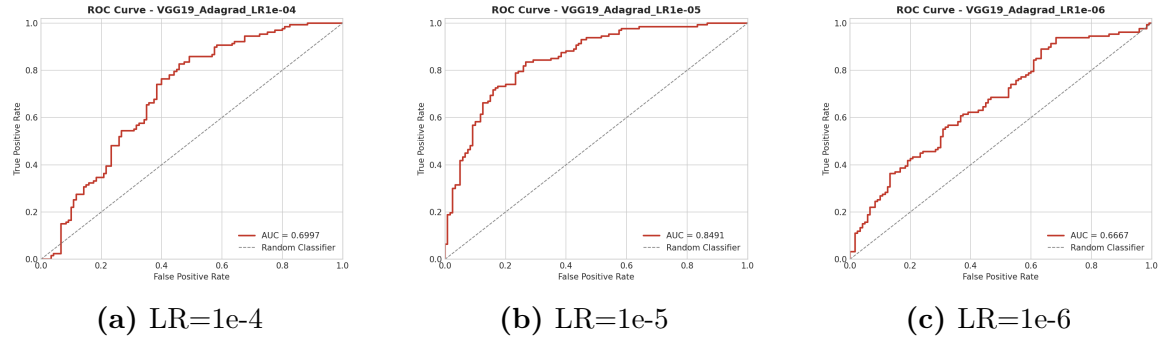


Figure 29: VGG19,Adagrad: AUC-ROC Curves across Learning Rates

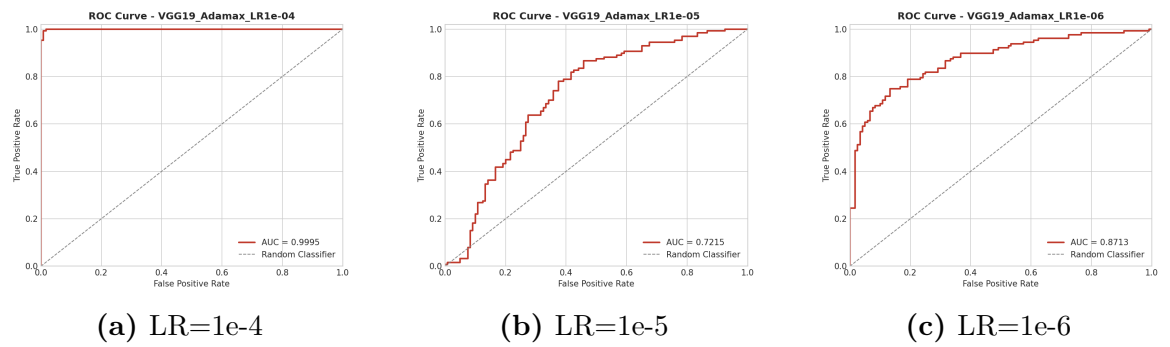


Figure 30: VGG19,Adamax: AUC-ROC Curves across Learning Rates

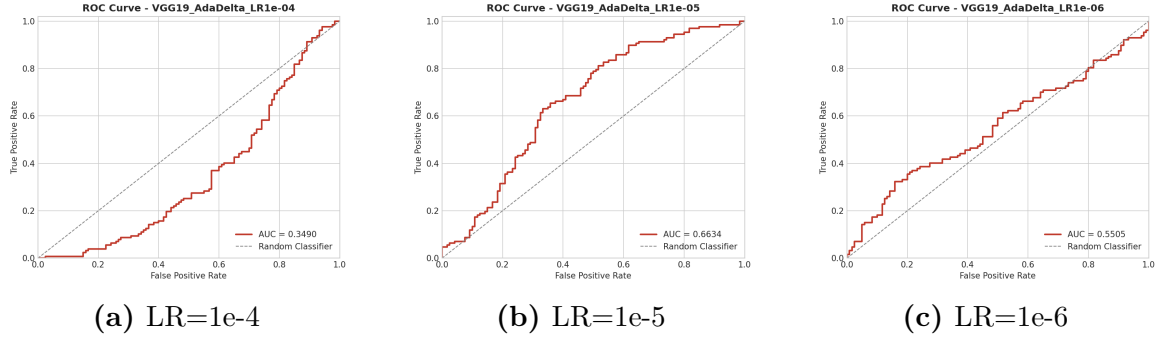


Figure 31: VGG19,AdaDelta: AUC-ROC Curves across Learning Rates

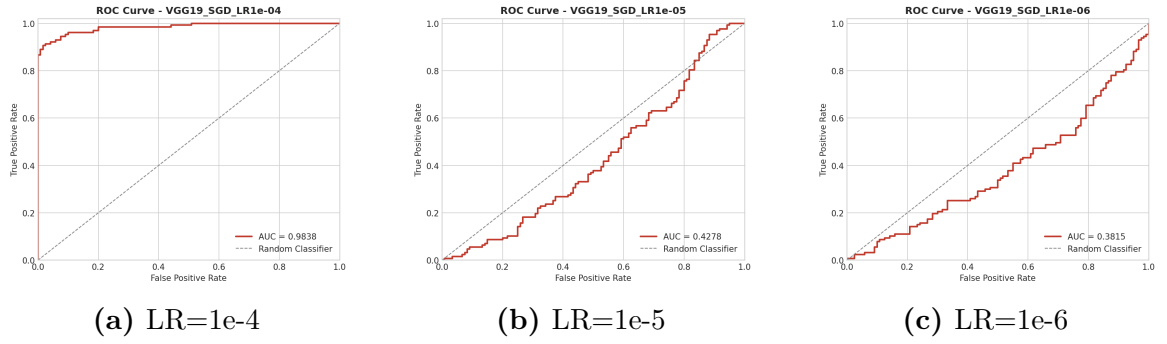


Figure 32: VGG19,SGD: AUC-ROC Curves across Learning Rates

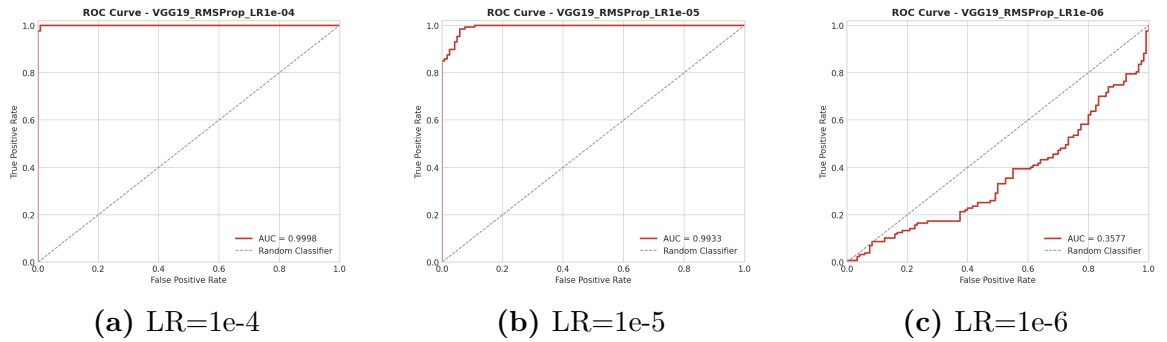
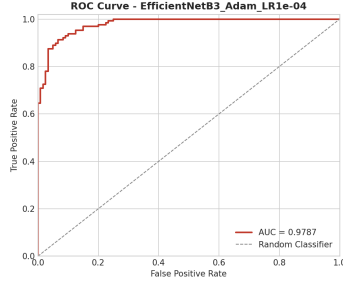
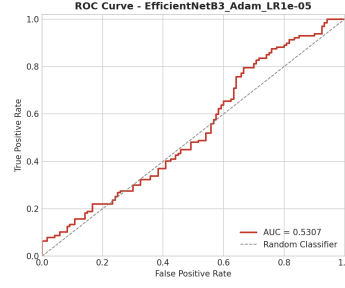


Figure 33: VGG19,RMSProp: AUC-ROC Curves across Learning Rates

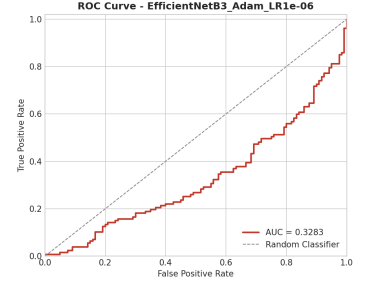
EfficientNetB3



(a) LR=1e-4

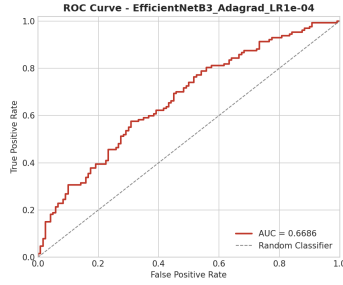


(b) LR=1e-5

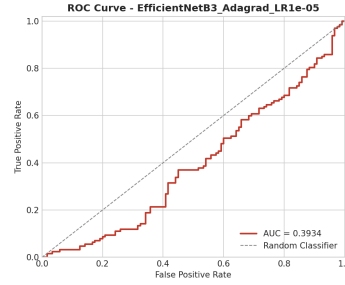


(c) LR=1e-6

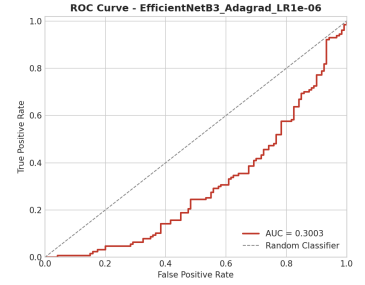
Figure 34: EfficientNetB3,Adam: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

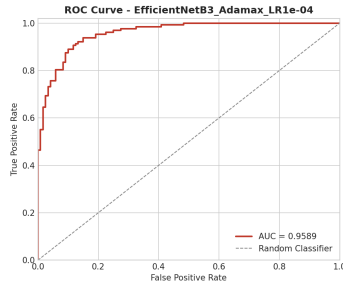


(b) LR=1e-5

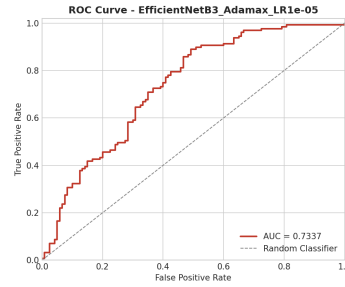


(c) LR=1e-6

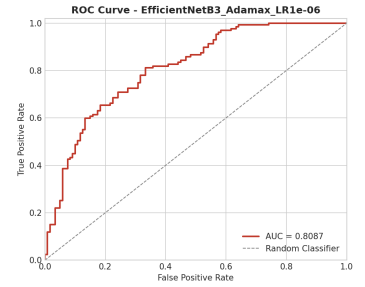
Figure 35: EfficientNetB3,Adagrad: AUC-ROC Curves across Learning Rates



(a) LR=1e-4



(b) LR=1e-5



(c) LR=1e-6

Figure 36: EfficientNetB3,Adamax: AUC-ROC Curves across Learning Rates

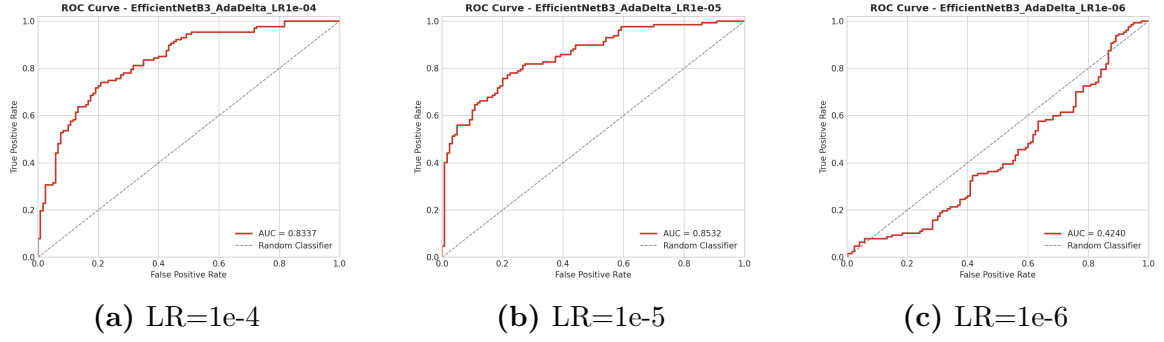


Figure 37: EfficientNetB3,AdaDelta: AUC-ROC Curves across Learning Rates

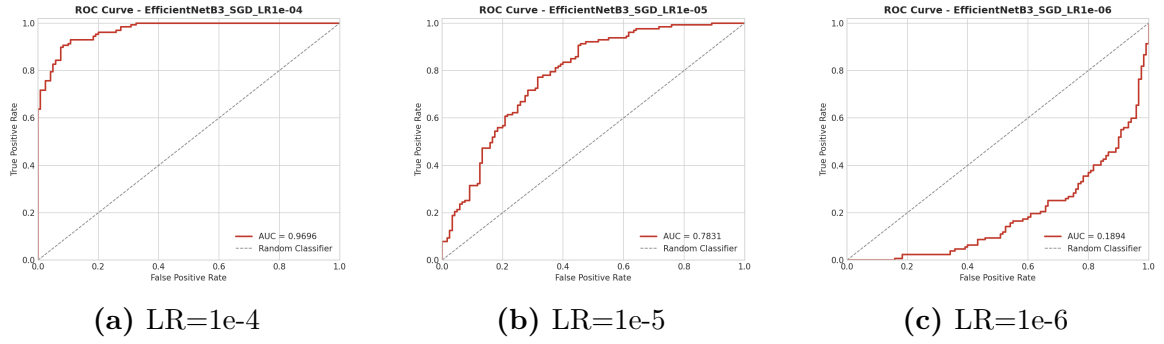


Figure 38: EfficientNetB3,SGD: AUC-ROC Curves across Learning Rates

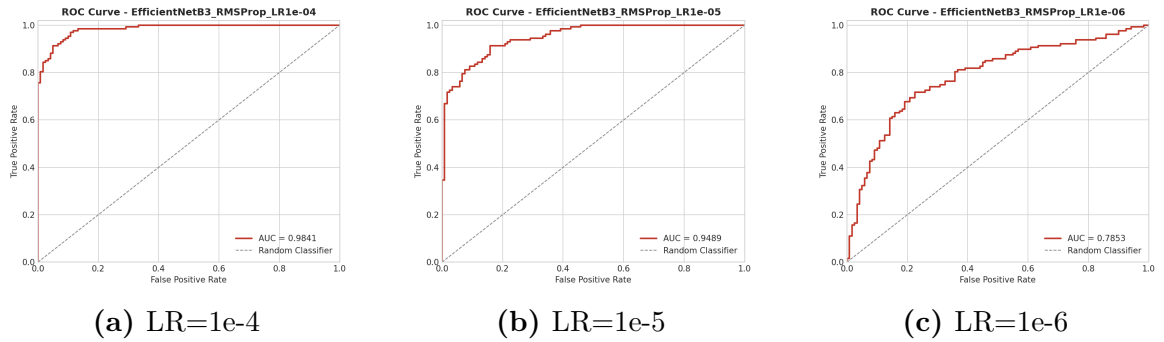


Figure 39: EfficientNetB3,RMSProp: AUC-ROC Curves across Learning Rates

ResNet50

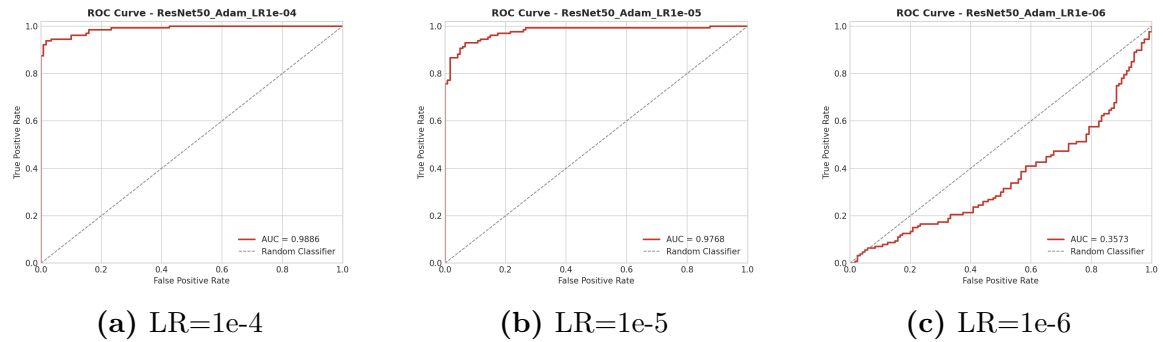


Figure 40: ResNet50,Adam: AUC-ROC Curves across Learning Rates

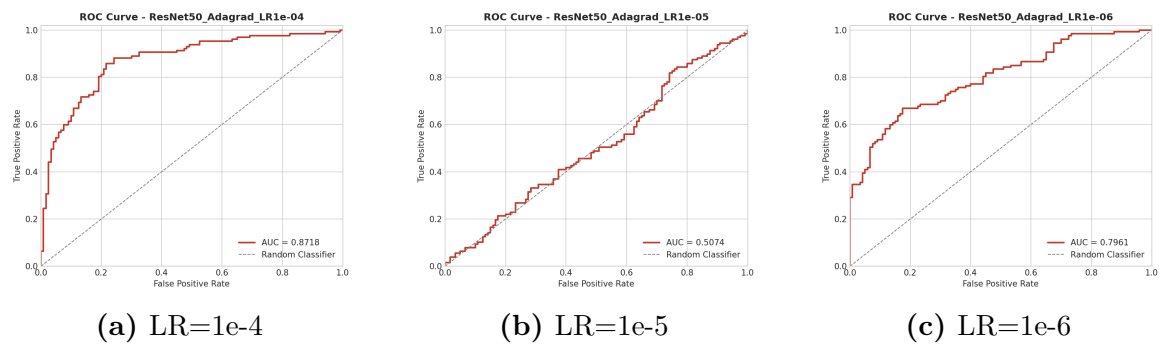


Figure 41: ResNet50,Adagrad: AUC-ROC Curves across Learning Rates

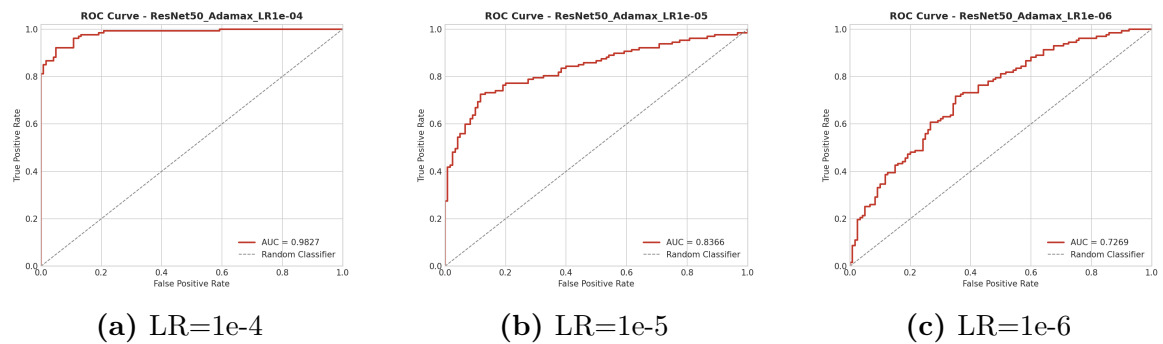


Figure 42: ResNet50,Adamax: AUC-ROC Curves across Learning Rates

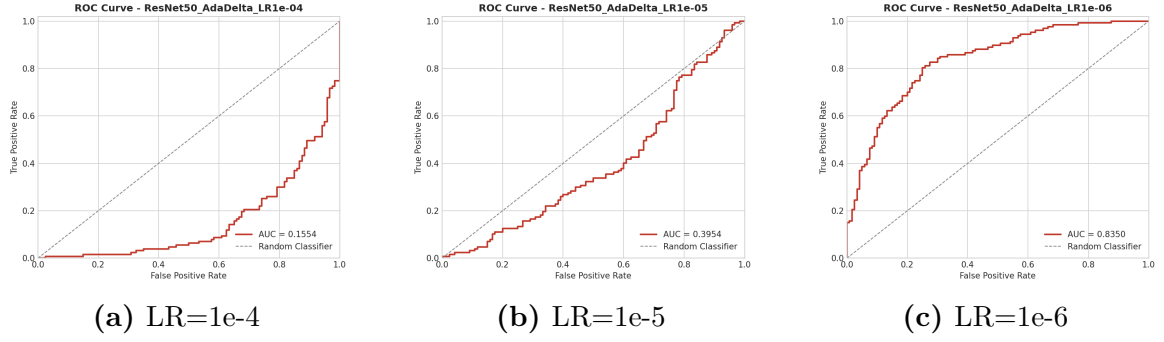


Figure 43: ResNet50,AdaDelta: AUC-ROC Curves across Learning Rates

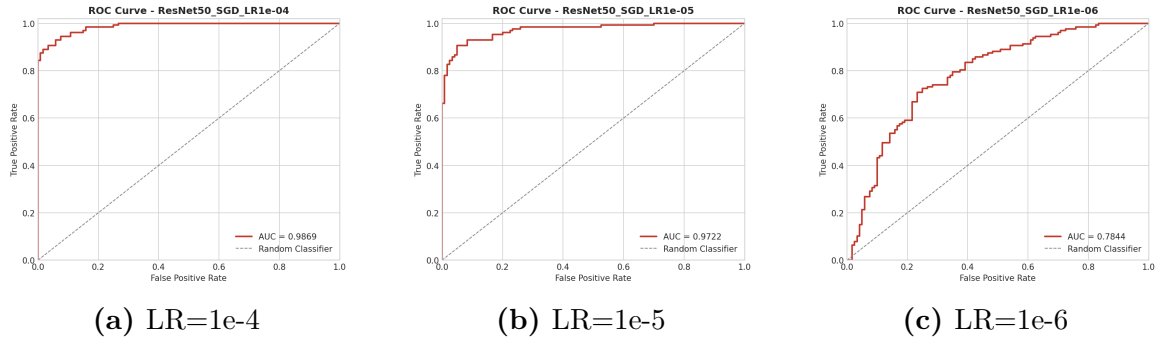


Figure 44: ResNet50,SGD: AUC-ROC Curves across Learning Rates

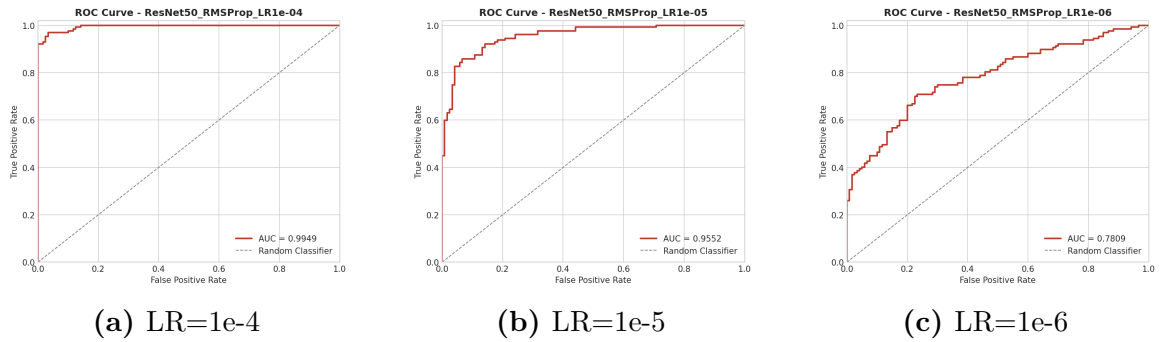
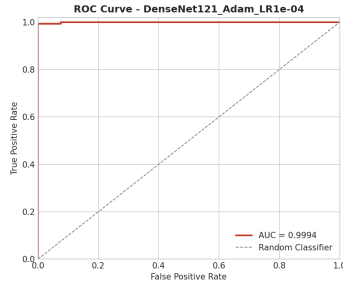
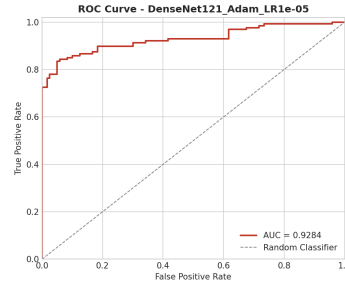


Figure 45: ResNet50,RMSProp: AUC-ROC Curves across Learning Rates

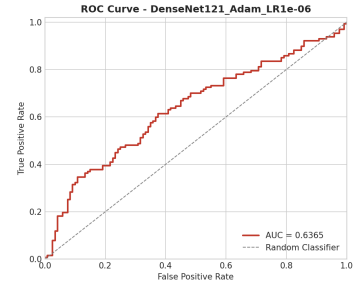
DenseNet121



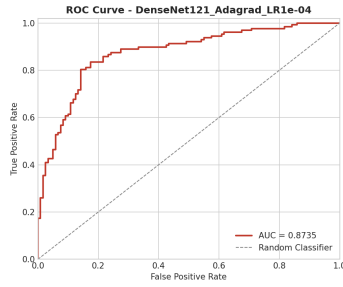
(a) LR=1e-4



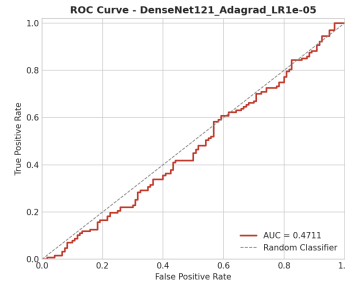
(b) LR=1e-5



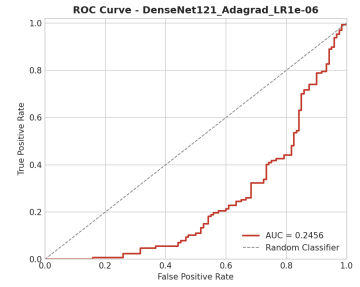
(c) LR=1e-6

Figure 46: DenseNet121,Adam: AUC-ROC Curves across Learning Rates

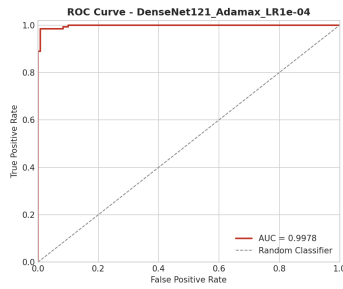
(a) LR=1e-4



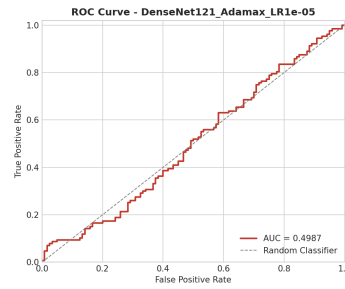
(b) LR=1e-5



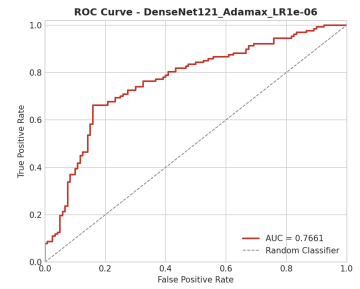
(c) LR=1e-6

Figure 47: DenseNet121,Adagrad: AUC-ROC Curves across Learning Rates

(a) LR=1e-4



(b) LR=1e-5



(c) LR=1e-6

Figure 48: DenseNet121,Adamax: AUC-ROC Curves across Learning Rates

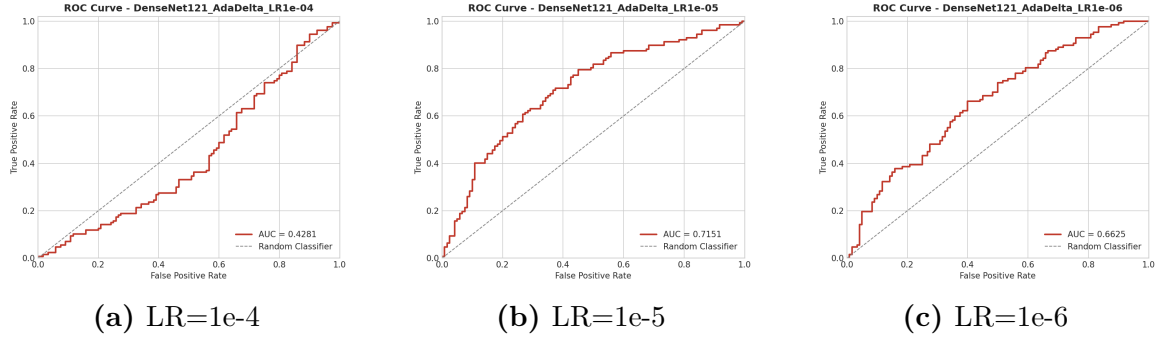


Figure 49: DenseNet121,AdaDelta: AUC-ROC Curves across Learning Rates

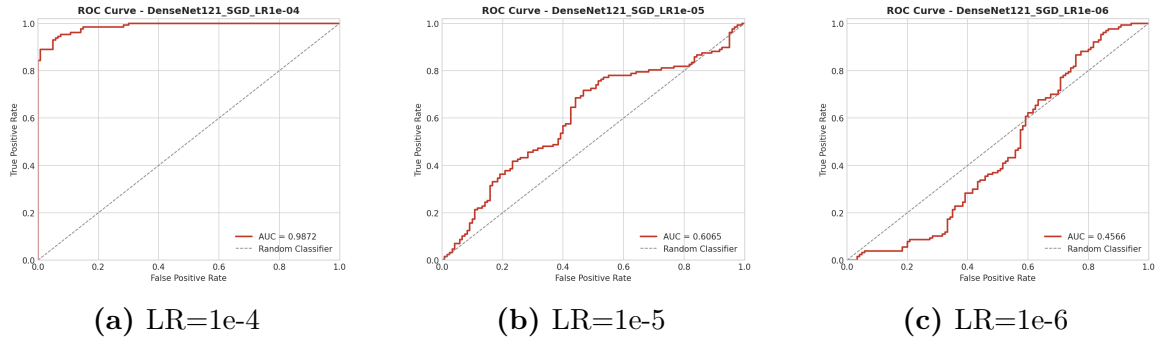


Figure 50: DenseNet121,SGD: AUC-ROC Curves across Learning Rates

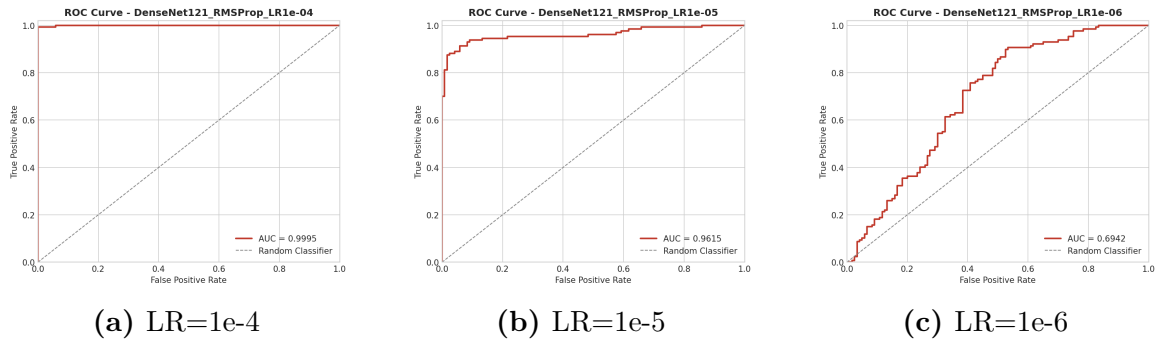


Figure 51: DenseNet121,RMSProp: AUC-ROC Curves across Learning Rates

9.3 Per-Model Confusion Matrices

VGG19

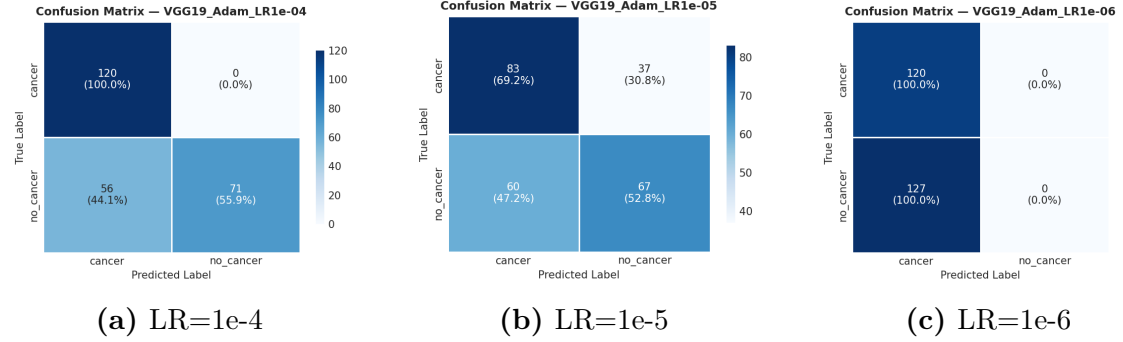


Figure 52: VGG19,Adam: Confusion Matrices across Learning Rates

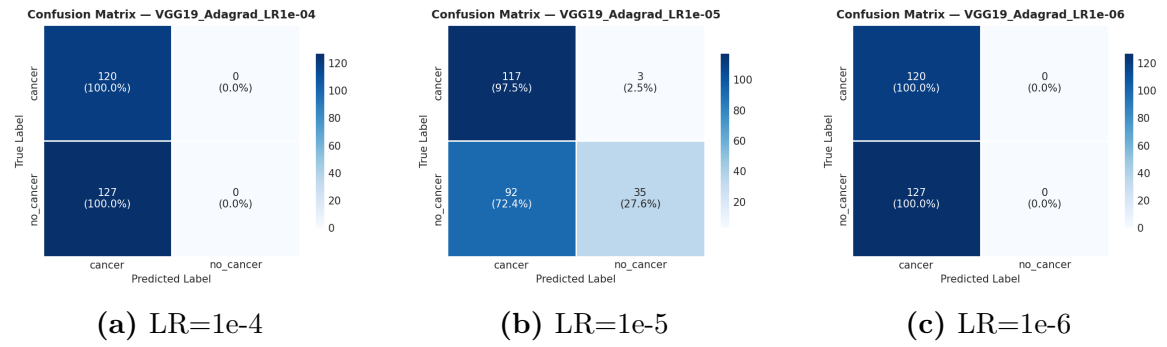


Figure 53: VGG19,Adagrad: Confusion Matrices across Learning Rates

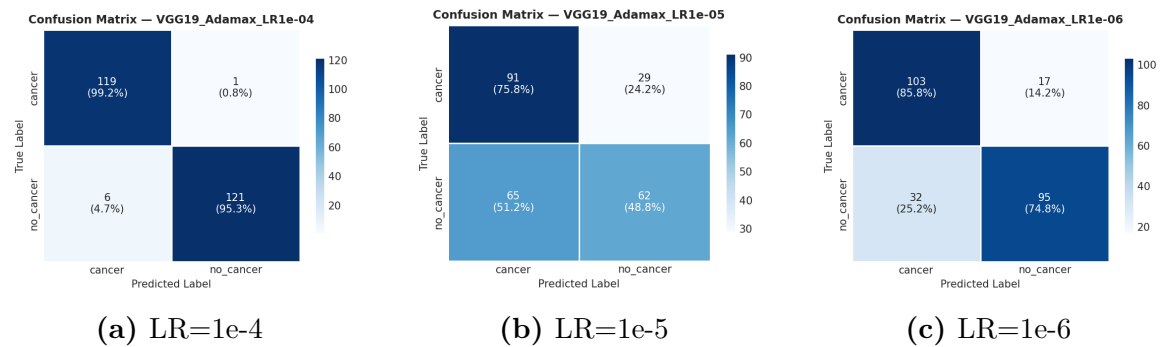


Figure 54: VGG19,Adamax: Confusion Matrices across Learning Rates

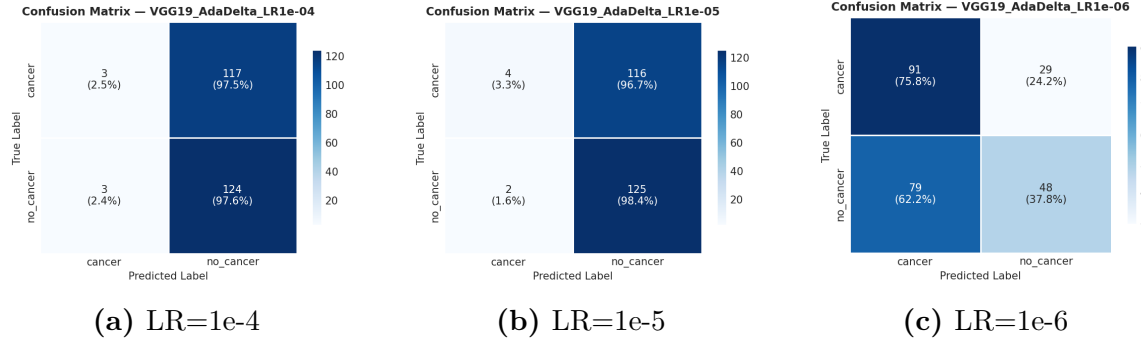


Figure 55: VGG19,AdaDelta: Confusion Matrices across Learning Rates

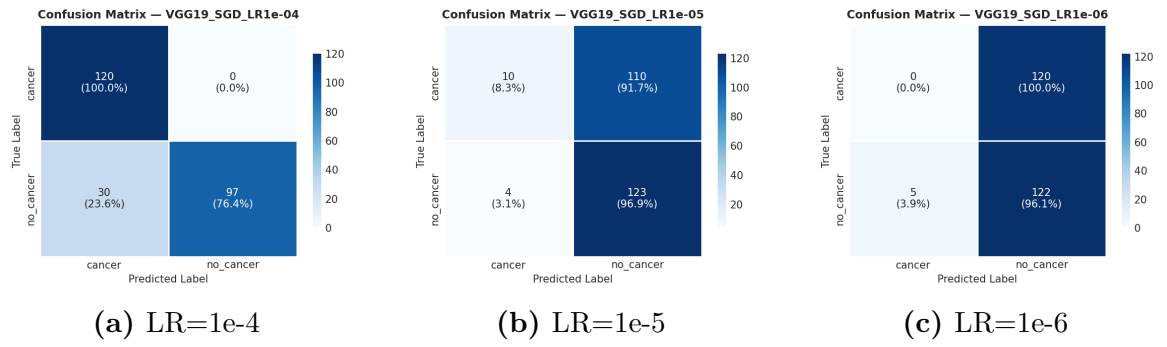


Figure 56: VGG19,SGD: Confusion Matrices across Learning Rates

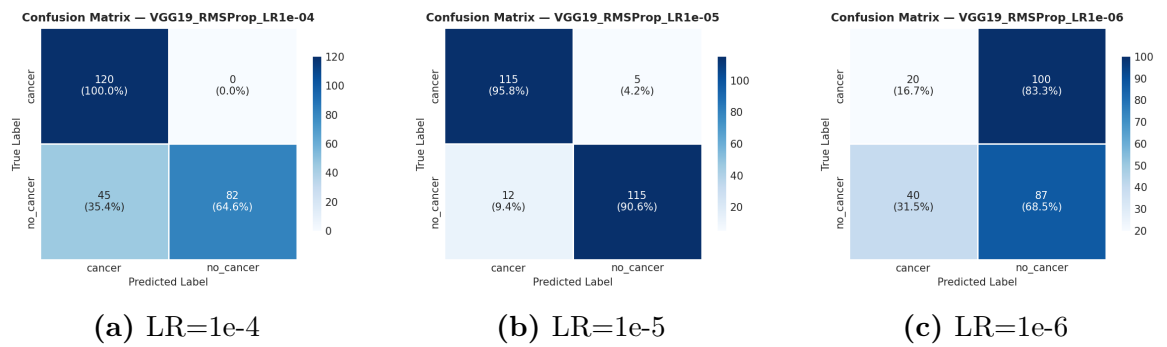
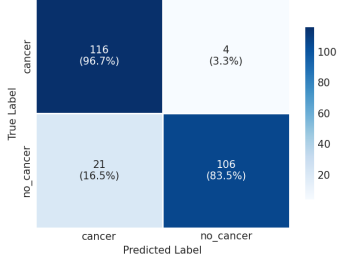


Figure 57: VGG19,RMSProp: Confusion Matrices across Learning Rates

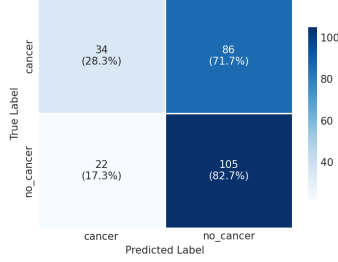
EfficientNetB3

Confusion Matrix — EfficientNetB3_Adam_LR1e-04



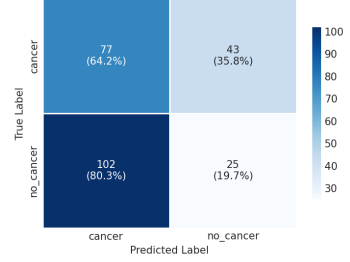
(a) LR=1e-4

Confusion Matrix — EfficientNetB3_Adam_LR1e-05



(b) LR=1e-5

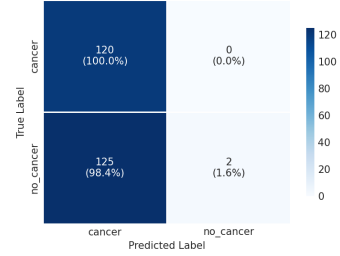
Confusion Matrix — EfficientNetB3_Adam_LR1e-06



(c) LR=1e-6

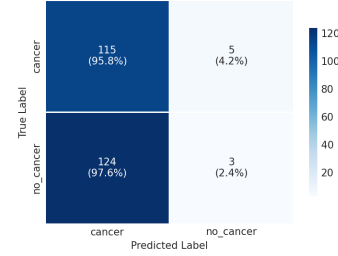
Figure 58: EfficientNetB3,Adam: Confusion Matrices across Learning Rates

Confusion Matrix — EfficientNetB3_Adagrad_LR1e-04



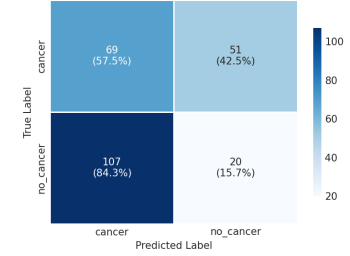
(a) LR=1e-4

Confusion Matrix — EfficientNetB3_Adagrad_LR1e-05



(b) LR=1e-5

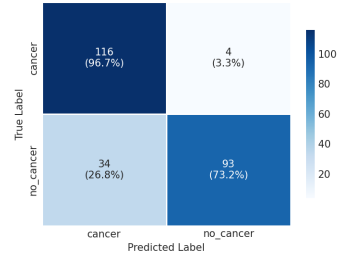
Confusion Matrix — EfficientNetB3_Adagrad_LR1e-06



(c) LR=1e-6

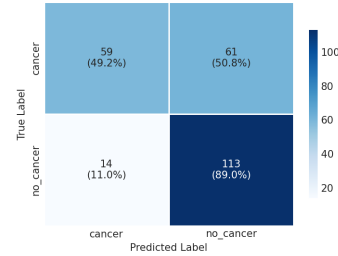
Figure 59: EfficientNetB3,Adagrad: Confusion Matrices across Learning Rates

Confusion Matrix — EfficientNetB3_Adamax_LR1e-04



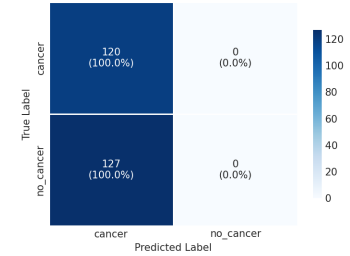
(a) LR=1e-4

Confusion Matrix — EfficientNetB3_Adamax_LR1e-05



(b) LR=1e-5

Confusion Matrix — EfficientNetB3_Adamax_LR1e-06



(c) LR=1e-6

Figure 60: EfficientNetB3,Adamax: Confusion Matrices across Learning Rates

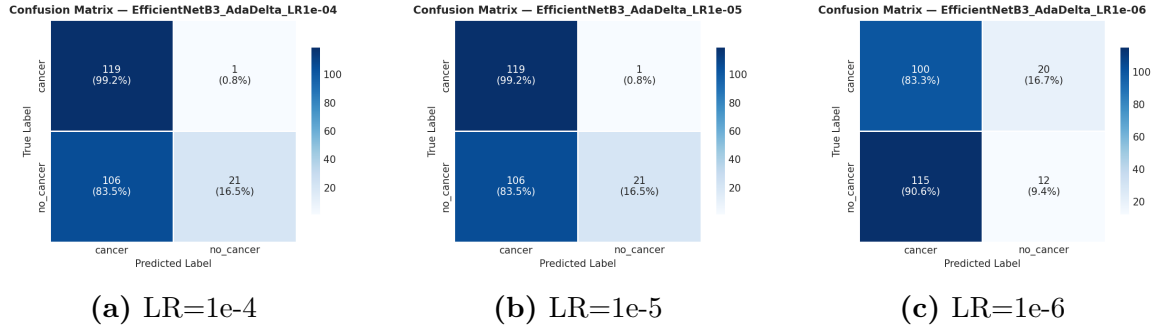


Figure 61: EfficientNetB3,AdaDelta: Confusion Matrices across Learning Rates

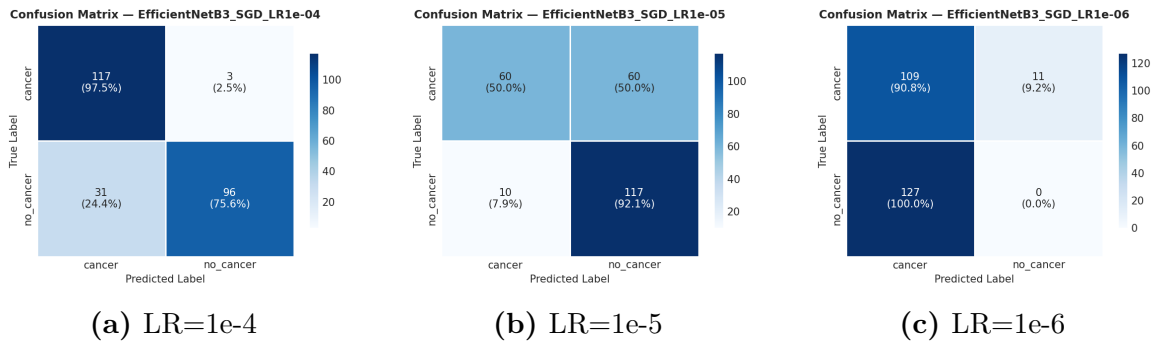


Figure 62: EfficientNetB3,SGD: Confusion Matrices across Learning Rates

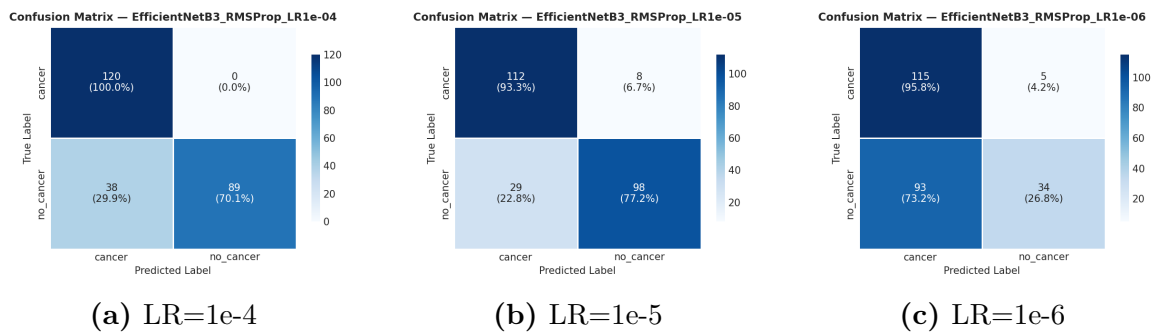


Figure 63: EfficientNetB3,RMSProp: Confusion Matrices across Learning Rates

ResNet50

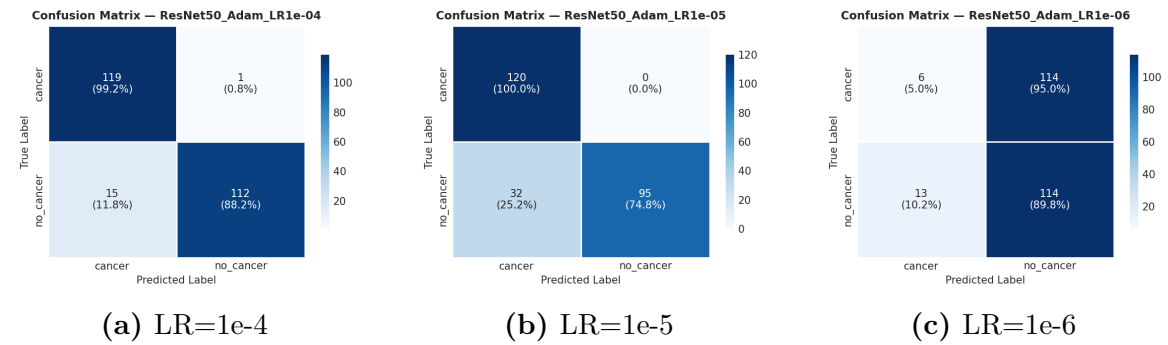


Figure 64: ResNet50,Adam: Confusion Matrices across Learning Rates

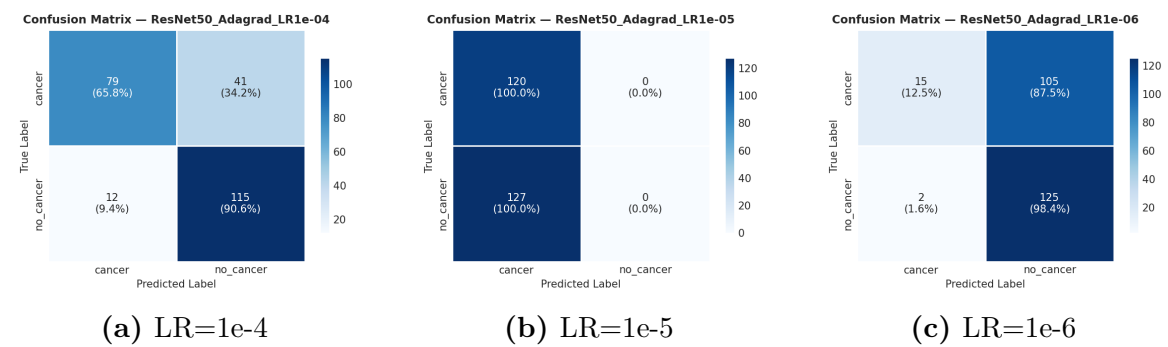


Figure 65: ResNet50,Adagrad: Confusion Matrices across Learning Rates

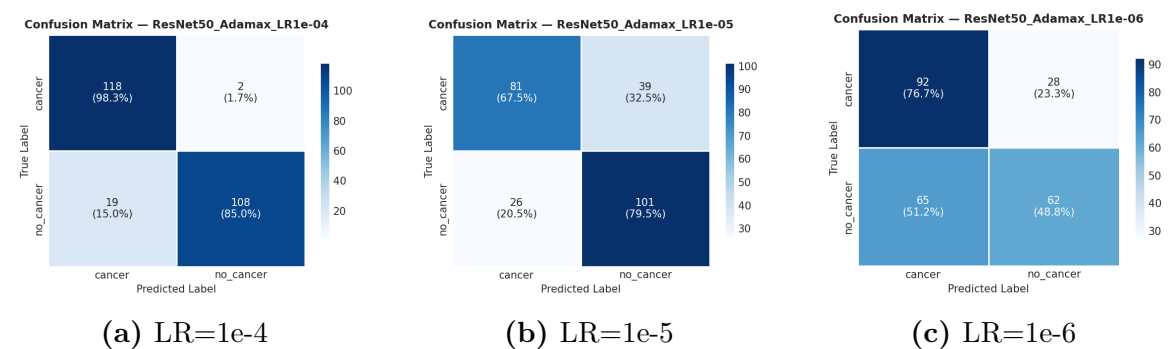


Figure 66: ResNet50,Adamax: Confusion Matrices across Learning Rates

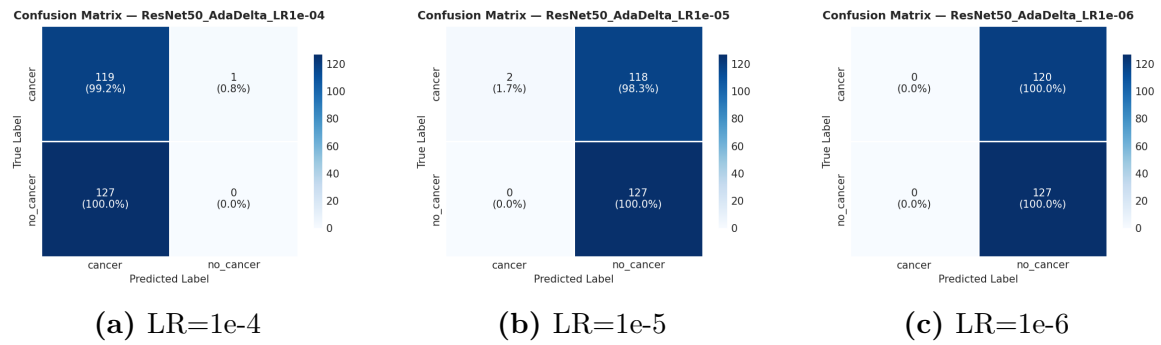


Figure 67: ResNet50,AdaDelta: Confusion Matrices across Learning Rates

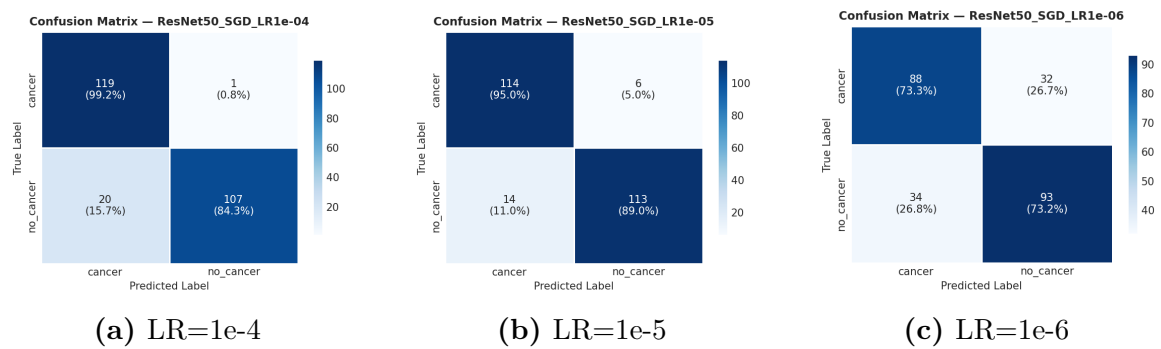


Figure 68: ResNet50,SGD: Confusion Matrices across Learning Rates

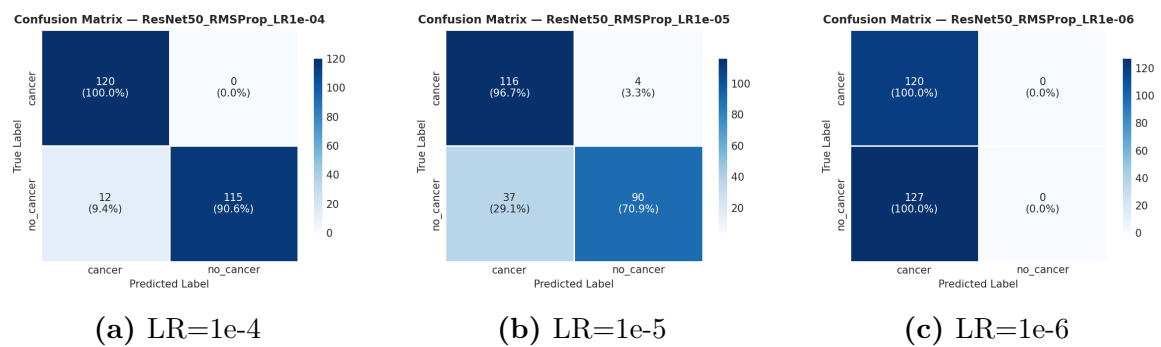


Figure 69: ResNet50,RMSProp: Confusion Matrices across Learning Rates

DenseNet121

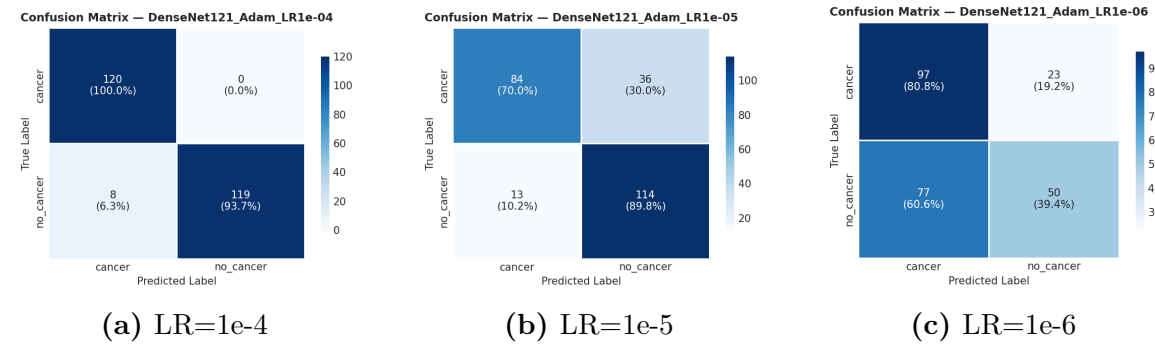


Figure 70: DenseNet121,Adam: Confusion Matrices across Learning Rates

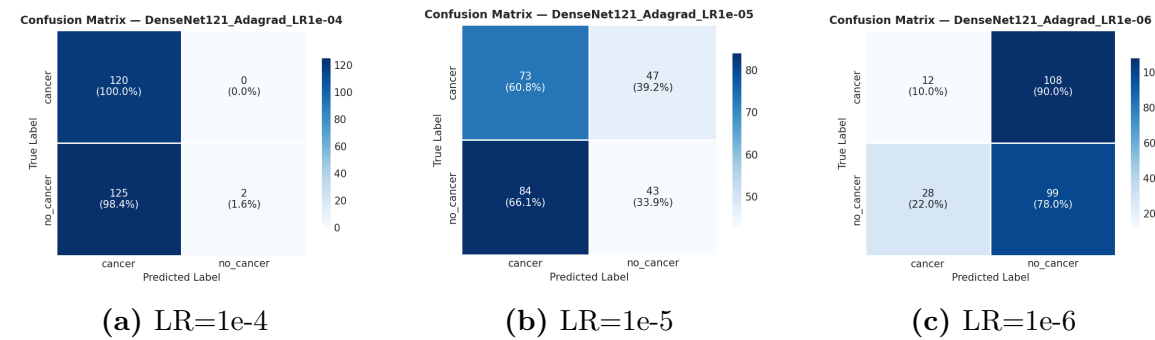


Figure 71: DenseNet121,Adagrad: Confusion Matrices across Learning Rates

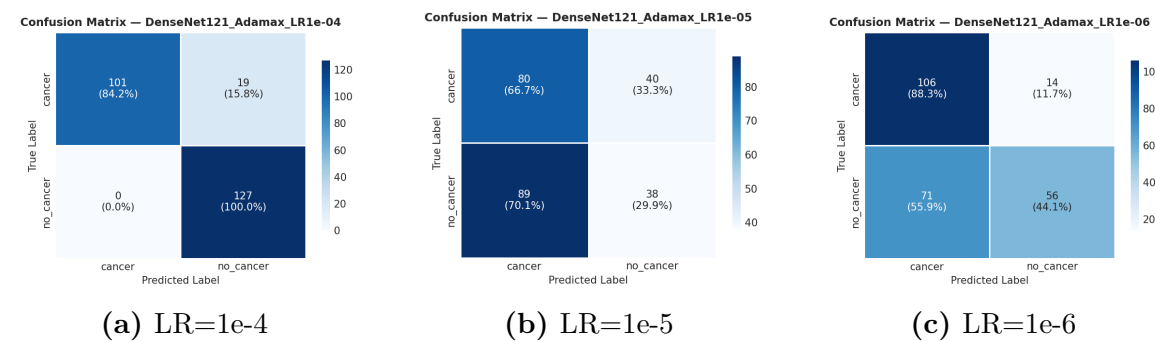


Figure 72: DenseNet121,Adamax: Confusion Matrices across Learning Rates

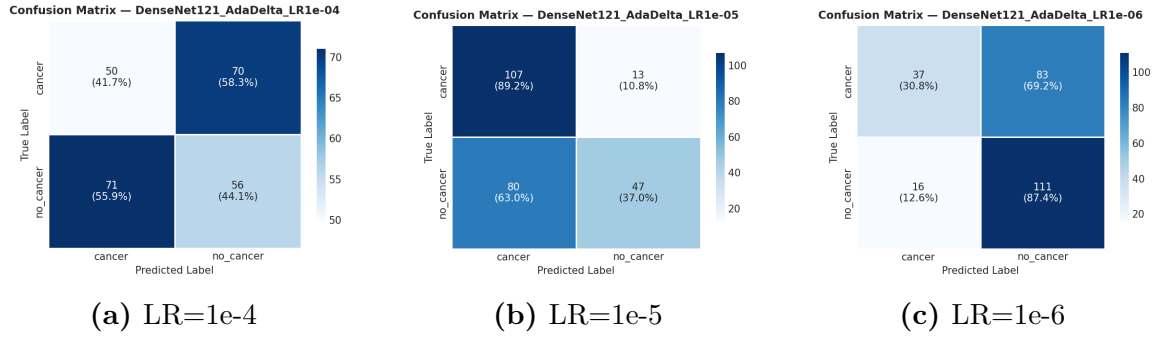


Figure 73: DenseNet121,AdaDelta: Confusion Matrices across Learning Rates

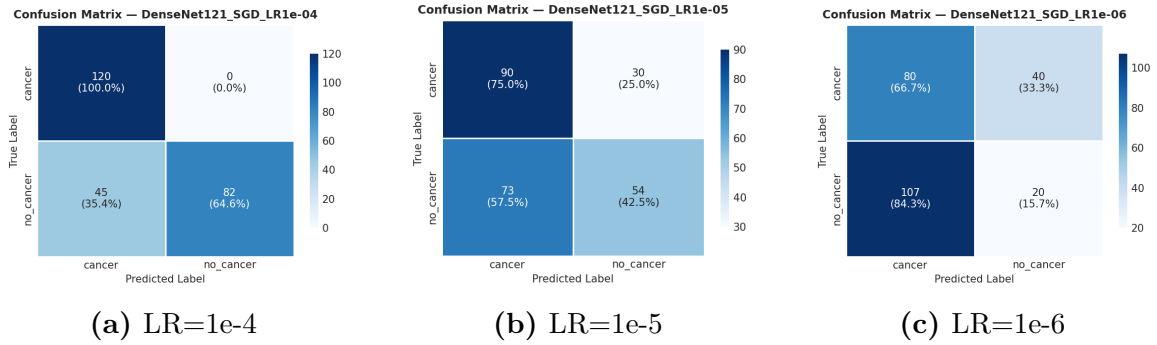


Figure 74: DenseNet121,SGD: Confusion Matrices across Learning Rates

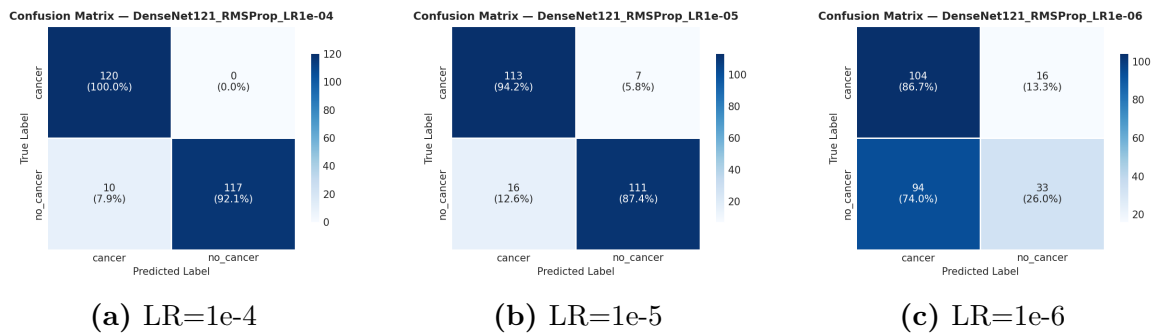


Figure 75: DenseNet121,RMSProp: Confusion Matrices across Learning Rates

9.4 Complete 72-Run Results Table

Table 7: Complete 72-run results table sorted by AUC-ROC descending.

Rk	Run Name	Arch	Opt	LR	Acc	Prec	Rec	Spec	F1	AUC	Ep
1	VGG19_Adam_LR1e-04	VGG19	Adam	1e-4	0.7733	1.0000	0.5591	1.0000	0.7172	0.9995	26
2	VGG19_Adamax_LR1e-04	VGG19	Adamax	1e-4	0.9717	0.9918	0.9528	0.9917	0.9719	0.9995	50
3	DenseNet121_Adam_LR1e-04	DenseNet121	Adam	1e-4	0.9676	1.0000	0.9370	1.0000	0.9675	0.9991	27
4	DenseNet121_RMSProp_LR1e-04	DenseNet121	RMSProp	1e-4	0.9595	1.0000	0.9213	1.0000	0.9590	0.9990	24
5	DenseNet121_Adamax_LR1e-04	DenseNet121	Adamax	1e-4	0.9231	0.8699	1.0000	0.8417	0.9304	0.9977	23
6	VGG19_RMSProp_LR1e-04	VGG19	RMSProp	1e-4	0.8178	1.0000	0.6457	1.0000	0.7847	0.9949	21
7	ResNet50_RMSProp_LR1e-04	ResNet50	RMSProp	1e-4	0.9514	1.0000	0.9055	1.0000	0.9504	0.9943	40
8	VGG19_RMSProp_LR1e-05	VGG19	RMSProp	1e-5	0.9312	0.9583	0.9055	0.9583	0.9312	0.9933	50
9	DenseNet121_SGD_LR1e-04	DenseNet121	SGD	1e-4	0.8178	1.0000	0.6457	1.0000	0.7847	0.9885	30
10	ResNet50_SGD_LR1e-04	ResNet50	SGD	1e-4	0.9150	0.9907	0.8425	0.9917	0.9106	0.9869	35
11	ResNet50_Adam_LR1e-04	ResNet50	Adam	1e-4	0.9352	0.9912	0.8819	0.9917	0.9333	0.9864	31
12	EfficientNetB3_RMSProp_LR1e-04	EfficientNetB3	RMSProp	1e-4	0.8462	1.0000	0.7008	1.0000	0.8241	0.9841	41
13	VGG19_SGD_LR1e-04	VGG19	SGD	1e-4	0.8785	1.0000	0.7638	1.0000	0.8661	0.9838	50
14	ResNet50_Adamax_LR1e-04	ResNet50	Adamax	1e-4	0.9150	0.9818	0.8504	0.9833	0.9114	0.9815	44
15	EfficientNetB3_Adam_LR1e-04	EfficientNetB3	Adam	1e-4	0.8988	0.9636	0.8346	0.9667	0.8945	0.9787	42
16	ResNet50_Adam_LR1e-05	ResNet50	Adam	1e-5	0.8704	1.0000	0.7480	1.0000	0.8559	0.9768	50
17	ResNet50_SGD_LR1e-05	ResNet50	SGD	1e-5	0.9190	0.9496	0.8898	0.9500	0.9187	0.9722	32
18	EfficientNetB3_SGD_LR1e-04	EfficientNetB3	SGD	1e-4	0.8623	0.9697	0.7559	0.9750	0.8496	0.9696	47
19	DenseNet121_RMSProp_LR1e-05	DenseNet121	RMSProp	1e-5	0.9069	0.9407	0.8740	0.9417	0.9061	0.9619	37
20	EfficientNetB3_Adamax_LR1e-04	EfficientNetB3	Adamax	1e-4	0.8462	0.9588	0.7323	0.9667	0.8304	0.9589	50
21	ResNet50_RMSProp_LR1e-05	ResNet50	RMSProp	1e-5	0.8340	0.9574	0.7087	0.9667	0.8145	0.9510	36
22	EfficientNetB3_RMSProp_LR1e-05	EfficientNetB3	RMSProp	1e-5	0.8502	0.9245	0.7717	0.9333	0.8412	0.9489	50
23	DenseNet121_Adam_LR1e-05	DenseNet121	Adam	1e-5	0.8016	0.7600	0.8976	0.7000	0.8231	0.9284	50
24	DenseNet121_Adagrad_LR1e-04	DenseNet121	Adagrad	1e-4	0.4939	1.0000	0.0157	1.0000	0.0310	0.8734	25
25	ResNet50_Adagrad_LR1e-04	ResNet50	Adagrad	1e-4	0.7854	0.7372	0.9055	0.6583	0.8127	0.8718	50
26	VGG19_Adamax_LR1e-06	VGG19	Adamax	1e-6	0.8016	0.8482	0.7480	0.8583	0.7950	0.8713	11
27	EfficientNetB3_AdaDelta_LR1e-05	EfficientNetB3	AdaDelta	1e-5	0.5668	0.9545	0.1654	0.9917	0.2819	0.8531	15
28	VGG19_Adagrad_LR1e-05	VGG19	Adagrad	1e-5	0.6154	0.9211	0.2756	0.9750	0.4242	0.8491	38
29	ResNet50_Adamax_LR1e-05	ResNet50	Adamax	1e-5	0.7368	0.7214	0.7953	0.6750	0.7566	0.8366	38
30	ResNet50_AdaDelta_LR1e-06	ResNet50	AdaDelta	1e-6	0.5142	0.5142	1.0000	0.0000	0.6791	0.8350	11
31	EfficientNetB3_AdaDelta_LR1e-04	EfficientNetB3	AdaDelta	1e-4	0.5668	0.9545	0.1654	0.9917	0.2819	0.8338	11
32	EfficientNetB3_Adamax_LR1e-06	EfficientNetB3	Adamax	1e-6	0.4858	0.0000	0.0000	1.0000	0.0000	0.8089	21
33	ResNet50_Adagrad_LR1e-06	ResNet50	Adagrad	1e-6	0.5668	0.5435	0.9843	0.1250	0.7003	0.7961	25
34	EfficientNetB3_RMSProp_LR1e-06	EfficientNetB3	RMSProp	1e-6	0.6032	0.8718	0.2677	0.9583	0.4096	0.7854	20
35	ResNet50_SGD_LR1e-06	ResNet50	SGD	1e-6	0.7328	0.7440	0.7323	0.7333	0.7381	0.7844	41
36	EfficientNetB3_SGD_LR1e-05	EfficientNetB3	SGD	1e-5	0.7166	0.6610	0.9213	0.5000	0.7697	0.7829	50
37	ResNet50_RMSProp_LR1e-06	ResNet50	RMSProp	1e-6	0.4858	0.0000	0.0000	1.0000	0.0000	0.7809	11
38	DenseNet121_Adamax_LR1e-06	DenseNet121	Adamax	1e-6	0.6559	0.8000	0.4409	0.8833	0.5685	0.7663	27
39	EfficientNetB3_Adamax_LR1e-05	EfficientNetB3	Adamax	1e-5	0.6964	0.6494	0.8898	0.4917	0.7508	0.7335	37
40	ResNet50_Adamax_LR1e-06	ResNet50	Adamax	1e-6	0.6235	0.6889	0.4882	0.7667	0.5714	0.7269	13
41	VGG19_Adamax_LR1e-05	VGG19	Adamax	1e-5	0.6194	0.6813	0.4882	0.7583	0.5688	0.7215	35
42	DenseNet121_AdaDelta_LR1e-05	DenseNet121	AdaDelta	1e-5	0.6235	0.7833	0.3701	0.8917	0.5027	0.7146	11
43	VGG19_Adam_LR1e-05	VGG19	Adam	1e-5	0.6073	0.6442	0.5276	0.6917	0.5801	0.7117	50
44	VGG19_Adagrad_LR1e-04	VGG19	Adagrad	1e-4	0.4858	0.0000	0.0000	1.0000	0.0000	0.6997	25
45	DenseNet121_RMSProp_LR1e-06	DenseNet121	RMSProp	1e-6	0.5547	0.6735	0.2598	0.8667	0.3750	0.6941	12
46	EfficientNetB3_Adagrad_LR1e-04	EfficientNetB3	Adagrad	1e-4	0.4939	1.0000	0.0157	1.0000	0.0310	0.6686	29
47	VGG19_Adagrad_LR1e-06	VGG19	Adagrad	1e-6	0.4858	0.0000	0.0000	1.0000	0.0000	0.6667	11
48	VGG19_AdaDelta_LR1e-05	VGG19	AdaDelta	1e-5	0.5223	0.5187	0.9843	0.0333	0.6793	0.6634	11
49	DenseNet121_AdaDelta_LR1e-06	DenseNet121	AdaDelta	1e-6	0.5992	0.5722	0.8740	0.3083	0.6916	0.6623	12
50	DenseNet121_Adam_LR1e-06	DenseNet121	Adam	1e-6	0.5951	0.6849	0.3937	0.8083	0.5000	0.6365	18
51	DenseNet121_SGD_LR1e-05	DenseNet121	SGD	1e-5	0.5830	0.6429	0.4252	0.7500	0.5118	0.6067	50
52	VGG19_Adam_LR1e-06	VGG19	Adam	1e-6	0.4858	0.0000	0.0000	1.0000	0.0000	0.5520	11
53	VGG19_AdaDelta_LR1e-06	VGG19	AdaDelta	1e-6	0.5628	0.6234	0.3780	0.7583	0.4706	0.5505	14
54	EfficientNetB3_Adam_LR1e-05	EfficientNetB3	Adam	1e-5	0.5628	0.5497	0.8268	0.2833	0.6604	0.5308	43
55	ResNet50_Adagrad_LR1e-05	ResNet50	Adagrad	1e-5	0.4858	0.0000	0.0000	1.0000	0.0000	0.5074	12
56	DenseNet121_Adamax_LR1e-05	DenseNet121	Adamax	1e-5	0.4777	0.4872	0.2992	0.6667	0.3707	0.4989	49
57	DenseNet121_Adagrad_LR1e-05	DenseNet121	Adagrad	1e-5	0.4696	0.4778	0.3386	0.6083	0.3963	0.4711	27
58	DenseNet121_SGD_LR1e-06	DenseNet121	SGD	1e-6	0.4049	0.3333	0.1575	0.6667	0.2139	0.4566	40
59	DenseNet121_AdaDelta_LR1e-04	DenseNet121	AdaDelta	1e-4	0.4291	0.4444	0.4409	0.4167	0.4427	0.4281	33
60	VGG19_SGD_LR1e-05	VGG19	SGD	1e-5	0.5385	0.5279	0.9685	0.0833	0.6833	0.4278	22
61	EfficientNetB3_AdaDelta_LR1e-06	EfficientNetB3	AdaDelta	1e-6	0.4534	0.3750	0.0945	0.8333	0.1509	0.4241	11
62	ResNet50_AdaDelta_LR1e-05	ResNet50	AdaDelta	1e-5	0.5223	0.5184	1.0000	0.0167	0.6828	0.3954	12
63	EfficientNetB3_Adagrad_LR1e-05	EfficientNetB3	Adagrad	1e-5	0.4777	0.3750	0.0236	0.9583	0.0444	0.3934	28
64	VGG19_SGD_LR1e-06	VGG19	SGD	1e-6	0.4939	0.5041	0.9606	0.0000	0.6612	0.3815	11
65	VGG19_RMSProp_LR1e-06	VGG19	RMSProp	1e-6	0.4332	0.4652	0.6850	0.1667	0.5541	0.3577	35
66	ResNet50_Adam_LR1e-06	ResNet50	Adam	1e-6	0.4858	0.5000	0.8976	0.0500	0.6423	0.3573	30
67	VGG19_AdaDelta_LR1e-04	VGG19	AdaDelta	1e-4	0.5142	0.5145	0.9764	0.0250	0.6739	0.3490	12
68	EfficientNetB3_Adam_LR1e-06	EfficientNetB3	Adam	1e-6	0.4130	0.3676	0.1969	0.6417	0.2564	0.3281	14
69	EfficientNetB3_Adagrad_LR1e-06	EfficientNetB3	Adagrad	1e-6	0.3603	0.2817	0.1575	0.5750	0.2020	0.3001	50
70	DenseNet121_Adagrad_LR1e-06	DenseNet121	Adagrad	1e-6	0.4494	0.4783	0.7795	0.1000	0.5928	0.2459	12
71	EfficientNetB3_SGD_LR1e-06	EfficientNetB3	SGD	1e-6	0.4413	0.0000	0.0000	0.9083	0.0000	0.1893	33
72	ResNet50_AdaDelta_LR1e-04	ResNet50	AdaDelta	1e-4	0.4818	0.0000	0.0000	0.9917	0.0000	0.1554	11

9.5 Source Code

src/preprocessing.py

Listing 1: preprocessing.py : Data Preprocessing Pipeline

```

1 """
2 preprocessing.py
3
4 Prepares the STRAMPN ovarian cancer
  dataset for training:
5   - Inventories raw images and
    verifies expected counts
6   - Performs a stratified train / val
    / test split (52.5% / 22.5% /
    25%)
7   - Copies files into dataset/
    processed/{split}/{class}/
    directories
8   - Prints a verification report and
    checks for cross-split leakage
9
10 Run directly: python src/
    preprocessing.py
11 """
12
13 import os
14 import shutil
15 import collections
16 from pathlib import Path
17
18 from sklearn.model_selection import
    train_test_split
19
20
21 # Directory layout
22
23 # Resolve paths relative to this file
    's location so the script works
    from
24 _PROJECT_ROOT = Path(__file__).
    resolve().parent.parent
25
26 _RAW_BASE = _PROJECT_ROOT / "dataset"
    / "raw" / "STRAMPN Dataset" / "
    Dataset"
27 _RAW_DIRS = {
28     "cancer": _RAW_BASE / "
    Ovarian_Cancer",
29     "no_cancer": _RAW_BASE / "
    Ovarian_Non_Cancer",
30 }
31
32 _PROCESSED_BASE = _PROJECT_ROOT / "
    dataset" / "processed"
33
34 _EXPECTED_COUNTS = {"cancer": 481, "
    no_cancer": 506}
35 _EXPECTED_TOTAL = 987
36
37
38 # Helpers
39
40 def _all_files(directory: Path) ->
    list[Path]:
41     """Return a sorted list of all
    files (non-recursive) in
    directory."""
42     return sorted(p for p in
    directory.iterdir() if p.
43
44
45     is_file())
46
47 def _copy_file(src: Path, dst_dir:
    Path) -> str:
48     """Copy src to dst_dir/src.name.
    Returns 'copied' or 'skipped'
    ,"""
49     dst = dst_dir / src.name
50     if dst.exists():
51         return "skipped"
52     shutil.copy2(src, dst)
53     return "copied"
54
55 # Main function
56 def run_preprocessing(random_seed:
    int - 42) -> None:
57     """
58     Full preprocessing pipeline:
59     inventory -> split -> copy ->
    verify.
60
61     Args:
62         random_seed: Controls the
63                     stratified split for
64                     reproducibility.
65                     Must be
66                     identical to
67                     the seed
68                     used in all
69                     84 training
70                     runs.
71
72     """
73
74     # Step 1 - Inventory
75     print("-" * 60)
76     print("STEP 1 - RAW DATASET
    INVENTORY")
77     print("-" * 60)
78
79     all_pairs: list[tuple[Path, str]]
80     = [] # (filepath, label)
81
82     for label, raw_dir in _RAW_DIRS.
83     items():
84         if not raw_dir.exists():
85             raise FileNotFoundError(
86                 f"Raw directory not
87                 found: {raw_dir}\n"
88                 "n"
89                 "Download the STRAMPN
90                 dataset and
91                 place it at:\n"
92                 f" {_RAW_BASE}")
93
94     files = _all_files(raw_dir)
95     non_jpg = [f for f in files
96                 if f.suffix.lower() != ".
97                 jpg"]
98
99     print(f"\n [{label}] path:
100           {raw_dir}")
101     print(f" Total files : {
102           len(files)}")
103
104     if non_jpg:
105         print(f" [WARNING] Non

```

```

-JPG files found ({
    len(non_jpg)}):")
87     for f in non_jpg:
88         print(f"          {f.
            name}")
89     else:
90         print(f"    All files    :
            .jpg [ok]")
91
92     jpg_files = [f for f in files
93                 if f.suffix.lower() == ".jpg"]
94     expected = _EXPECTED_COUNTS[
95                 label]
96     match_sym = "[ok]" if len(
97                 jpg_files) == expected
98     else "[x] (expected {ex
99                 pected})"
100     print(f"    JPG count    : {
101             len(jpg_files)} {
102             match_sym}")
103
104     all_pairs.extend((f, label)
105                     for f in jpg_files)
106
107     total = len(all_pairs)
108     total_sym = "[ok]" if total ==
109                 _EXPECTED_TOTAL else f"[x] (e
110                 xpected {_EXPECTED_TOTAL})"
111     print(f"\n Total images across
112           both classes: {total} {
113           total_sym}")
114
115     # Step 2 - Stratified Split
116     print("\n" + "-" * 60)
117     print("STEP 2 - STRATIFIED SPLIT"
118           )
119     print("-" * 60)
120
121     filepaths = [p for p, _ in
122                  all_pairs]
123     labels = [l for _, l in
124               all_pairs]
125
126     # First split: 75% train+val, 25%
127     test
128     paths_trainval, paths_test,
129     labels_trainval, labels_test
130     = train_test_split(
131         filepaths, labels,
132         test_size=0.25,
133         random_state=random_seed,
134         stratify=labels,
135     )
136
137     # Second split: 70% train, 30%
138     val (of the 75% slice ->
139     ~52.5% / 22.5% overall)
140     paths_train, paths_val,
141     labels_train, labels_val =
142     train_test_split(
143         paths_trainval,
144         labels_trainval,
145         test_size=0.30,
146         random_state=random_seed,
147         stratify=labels_trainval,
148     )
149
150     splits = {
151         "train": (paths_train,
152                  labels_train),
153         "val": (paths_val,
154                labels_val),
155         "test": (paths_test,
156                  labels_test),
157     }
158
159     print(f"\n Seed: {random_seed}")
160     print(f" {'Split':<8} {'cancer
161           ':>8} {'no_cancer':>10} {'
162           total':>7}")
163     print(f" {'-'*8} {'-'*8} {'-
164           '*10} {'-'*7}")
165     for split_name, (paths, lbls) in
166         splits.items():
167         c = lbls.count("cancer")
168         nc = lbls.count("no_cancer")
169         print(f" {split_name:<8} {c
170               :>8} {nc:>10} {len(lbls
171               ):>7}")
172     print(f" {'total':<8} {labels.
173           count('cancer'):>8} {labels.
174           count('no_cancer'):>10} {
175           total:>7}")
176
177     # Step 3 - Copy Files
178     print("\n" + "-" * 60)
179     print("STEP 3 - COPYING FILES TO
180           dataset/processed/")
181     print("-" * 60)
182
183     for split_name, (paths, lbls) in
184         splits.items():
185         copied_total = 0
186         skipped_total = 0
187
188         for src, label in zip(paths,
189                               lbls):
190             dst_dir = _PROCESSED_BASE
191                     / split_name / label
192             dst_dir.mkdir(parents=
193                           True, exist_ok=True)
194             result = _copy_file(src,
195                                 dst_dir)
196             if result == "copied":
197                 copied_total += 1
198             else:
199                 skipped_total += 1
200
201         print(
202             f" {split_name:<6}
203               copied: {copied_total
204               :>4} "
205             f"skipped (already exist)
206               : {skipped_total:>4}")
207
208     # Step 4 - Verification Report
209     print("\n" + "-" * 60)
210     print("STEP 4 - VERIFICATION
211           REPORT")
212     print("-" * 60)
213
214     class_names = ["cancer", "
215                     no_cancer"]
216     grand_total = 0
217
218     print(f"\n {'Split':<8} {'Class
219           ':<12} {'Count':>6} {'% of
220           total':>10}")
221     print(f" {'-'*8} {'-'*12} {'-
222           '*6} {'-'*10}")
223     for split_name in ["train", "val"
224                       , "test"]:
```

```

178         for cls in class_names:
179             cls_dir = _PROCESSED_BASE
180                 / split_name / cls
181             n = len(list(cls_dir.glob
182                 ("*.jpg"))) if
183                 cls_dir.exists() else
184                 0
185             pct = n / total * 100
186             grand_total += n
187             print(f" {split_name:<8}
188                 {cls:<12} {n:>6}
189                 {pct:>9.1f}%")
190
191     print(f"\n Grand total in
192         processed/: {grand_total} "
193         f"({'[ok] matches raw' if
194             grand_total == total
195             else '[x] MISMATCH with
196             raw'})")
197
198     # Cross-split leakage check -
199     # compare stem names across
200     # splits
201     print("\n Checking for cross-
202         split filename leakage.")
203     split_stems: dict[str, set[str]]
204         = {}
205     for split_name in ["train", "val"
206         , "test"]:
207         stems: set[str] = set()
208         for cls in class_names:
209             cls_dir = _PROCESSED_BASE
210                 / split_name / cls
211             if cls_dir.exists():
212                 stems.update(p.stem
213                     for p in cls_dir.
214                     glob("*.jpg"))
215     split_stems[split_name] =
216         stems
217
218     violations: list[str] = []
219     split_list = list(split_stems.
220         items())
221     for i in range(len(split_list)):
222         for j in range(i + 1, len(
223             split_list)):
224             a_name, a_stems =
225                 split_list[i]
226             b_name, b_stems =
227                 split_list[j]
228             overlap = a_stems &
229                 b_stems
230             if overlap:
231                 violations.append(
232                     f" [x] {a_name
233                         } intersect {
234                         b_name}: {len
235                         (overlap)}
236                         shared
237                         filename(s):
238                         "
239                     + ", ".join(
240                         sorted(
241                             overlap)[:5])
242                     + (".." if len(
243                         overlap) > 5
244                         else ""))
245             )
246
247     print()
248     if violations:
249         print(" [WARNING] INTEGRITY
250             VIOLATIONS FOUND:")

```

```

216         for v in violations:
217             print(v)
218         else:
219             print(" PREPROCESSING
220                 COMPLETE - No integrity
221                 violations found")
222
223     print("\n" + "-" * 60)
224
225     # Entry point
226
227     if __name__ == "__main__":
228         run_preprocessing()

```

src/model_builder.py

Listing 2: model_builder.py : Model Architecture Builder

```

1  """
2  model_builder.py
3
4  Builds compiled Keras transfer-
5  learning models for the 72-
6  simulation experiment matrix.
7
8  """
9
10 import tensorflow as tf
11 from tensorflow.keras import Model
12 from tensorflow.keras.applications
13     import VGG19, EfficientNetB3,
14     ResNet50, DenseNet121
15 from tensorflow.keras.layers import (
16     GlobalAveragePooling2D, Dense,
17     Dropout, BatchNormalization,
18 )
19 from tensorflow.keras.metrics import
20     Precision, Recall, AUC
21 from tensorflow.keras.optimizers
22     import (
23         Adam, Adagrad, Adamax, Adadelta,
24         SGD, RMSprop,
25 )
26
27 # Architecture registry
28 _ARCHITECTURE_REGISTRY = {
29     "VGG19": VGG19,
30     "EfficientNetB3": EfficientNetB3,
31     ,
32     "ResNet50": ResNet50,
33     "DenseNet121": DenseNet121,
34 }
35
36 # Optimizer registry
37 def _make_optimizer(name: str,
38     learning_rate: float):
39     """Instantiate an optimizer by
40         name with the given learning
41         rate."""
42
43     registry = {
44         "Adam": lambda lr: Adam(
45             learning_rate=lr),
46         "Adagrad": lambda lr:
47             Adagrad(learning_rate=lr)
48         ,
49         "Adamax": lambda lr: Adamax
50             (learning_rate=lr),
51         "AdaDelta": lambda lr:
52             Adadelta(learning_rate=lr
53             ),

```

```

35         "SGD": lambda lr: SGD(
36             learning_rate=lr,
37             momentum=0.9),
38         "RMSProp": lambda lr:
39             RMSprop(learning_rate=lr)
40     },
41     if name not in registry:
42         raise ValueError(
43             f"Unknown optimizer '{
44                 name}'".
45             f"Choose from: {sorted(
46                 registry.keys())}")
47     )
48     return registry[name](
49         learning_rate)
50
51 # Model builder
52 def build_model(
53     architecture_name: str,
54     optimizer_name: str,
55     learning_rate: float,
56     input_shape: tuple - (256, 256,
57         3),
58 ) -> Model:
59     """
60     Build and compile a transfer-
61     learning model for binary
62     ovarian cancer classification
63     .
64
65     Args:
66         architecture_name: One of '
67             VGG19', 'EfficientNetB3',
68             'ResNet50', 'DenseNet121'
69             .
70         optimizer_name: One of '
71             Adam', 'Adagrad', 'Adamax'
72             ', 'AdaDelta',
73             'SGD', '
74             RMSProp'
75             .
76         learning_rate: Floating-
77             point learning rate (e.g.
78             1e-4).
79         input_shape: HWC tuple;
80             must match the
81             generators (default 256x
82             256x3).
83
84     Returns:
85         A compiled tf.keras.Model
86         ready for model.fit().
87     """
88     if architecture_name not in
89         _ARCHITECTURE_REGISTRY:
90         raise ValueError(
91             f"Unknown architecture '{
92                 architecture_name}'".
93             f"Choose from: {sorted(
94                 _ARCHITECTURE_REGISTRY
95                 .keys())}")
96     )
97
98     # Base model (frozen ImageNet
99     weights)
100     base_cls = _ARCHITECTURE_REGISTRY
101         [architecture_name]
102     base_model = base_cls(
103         include_top=False,
104         weights="imagenet",
105         input_shape=input_shape,
106     )
107     base_model.trainable = False
108
109     # Custom classification head
110     x = base_model.output
111     x = GlobalAveragePooling2D()(x)
112     x = Dense(256, activation="relu")(
113         x)
114     x = BatchNormalization()(x)
115     x = Dropout(0.6)(x)
116     x = Dense(128, activation="relu")(
117         x)
118     x = BatchNormalization()(x)
119     x = Dropout(0.4)(x)
120     x = Dense(64, activation="relu")(
121         x)
122     x = BatchNormalization()(x)
123     x = Dropout(0.3)(x)
124     output = Dense(1, activation="
125         sigmoid")(x)
126     model = Model(inputs=base_model.
127         input, outputs=output)
128
129     # Compile
130     optimizer = _make_optimizer(
131         optimizer_name, learning_rate
132     )
133     model.compile(
134         optimizer=optimizer,
135         loss="binary_crossentropy",
136         metrics=[
137             "accuracy",
138             Precision(name="precision"
139             ),
140             Recall(name="recall"),
141             AUC(name="auc"),
142         ],
143     )
144     return model

```

src/train.py

Listing 3: train.py : Model Training Loop

```

1  """
2  train.py
3
4  Runs the training loop for a single
5  model configuration (one of the
6  72 runs).
7  Called once per simulation from
8  run_all.py or 02_experiments.
9  ipynb.
10
11 Each run writes the following
12 artifacts to results/{run_name}/:
13
14 - best_model.keras
15 - best
16   checkpoint by val_auc
17 - training_log.csv
18 - per-
19   epoch metrics
20 - {run_name}_acc_loss.png
21   - accuracy and
22   loss curves

```

```

12 - {run_name}_auc_roc.png
    - AUC-ROC curve
13 - confusion_matrices/{run_name}.png
    - confusion matrix heatmap
14 - (metrics row appended to master
    CSV by the caller)
15 """
16
17 import pathlib
18
19 import tensorflow as tf
20 from tensorflow.keras.callbacks
    import (
21     CSVLogger,
22     EarlyStopping,
23     ModelCheckpoint,
24     ReduceLROnPlateau,
25 )
26
27 from src.evaluate import
    evaluate_model
28 from src.visualize import
    plot_accuracy_loss, plot_auc_roc
29
30
31 def train_model(
32     model: tf.keras.Model,
33     train_generator,
34     val_generator,
35     test_generator,
36     architecture: str,
37     optimizer_name: str,
38     learning_rate: float,
39     results_base_dir,
40     epochs: int - 50,
41 ) -> dict:
42     """
43     Train one model configuration,
44     evaluate on the test set, and
45     save all artifacts.
46
47     Args:
48         model: Compiled
49             Keras model from
50             model_builder.build_model
51             ().
52         train_generator: Training
53             data generator (shared,
54             not re-instantiated).
55         val_generator: Validation
56             data generator.
57         test_generator: Test data
58             generator.
59         architecture: Base
60             architecture name (e.g. '
61             ResNet50').
62         optimizer_name: Optimizer
63             name (e.g. 'Adam').
64         learning_rate: Learning
65             rate float (e.g. 1e-4).
66         results_base_dir: Path-like;
67             run output written to
68             results_base_dir/run_name
69             /.
70         epochs: Maximum
71             number of training epochs
72             .
73
74     Returns:
75         metrics dict as returned by
76         evaluate_model (one row
77         of the master CSV).
78     """
79
80     results_base_dir = pathlib.Path(
81         results_base_dir)
82
83     # Step 1 - Setup
84     lr_str = f"{learning_rate:.0e}"
85         # e.g. '1
86         e-04'
87     run_name = f"{architecture}_{
88         optimizer_name}_LR{lr_str}"
89     run_dir = results_base_dir /
90         run_name
91     run_dir.mkdir(parents=True, ex
92         ist_ok=True)
93
94     print("-" * 60)
95     print(f"TRAINING: {run_name}")
96     print(f"Architecture : {
97         architecture}")
98     print(f"Optimizer : {
99         optimizer_name}")
100     print(f"Learning Rate: {
101         learning_rate}")
102     print("-" * 60)
103
104     # Step 2 - Callbacks
105     callbacks = [
106         ModelCheckpoint(
107             filepath=str(run_dir / "
108                 best_model.keras"),
109             monitor="val_auc",
110             mode="max",
111             save_best_only=True,
112             verbose=0,
113         ),
114         EarlyStopping(
115             monitor="val_auc",
116             patience=10,
117             mode="max",
118             restore_best_weights=True
119             ,
120             min_delta=0.001,
121             verbose=1,
122         ),
123         CSVLogger(
124             filename=str(run_dir / "
125                 training_log.csv"),
126             append=False,
127         ),
128         ReduceLROnPlateau(
129             monitor="val_loss",
130             factor=0.5,
131             patience=5,
132             min_lr=1e-7,
133             verbose=1,
134         ),
135     ]
136
137     # Step 3 - Training
138     history = model.fit(
139         train_generator,
140         validation_data=val_generator
141         ,
142         epochs=epochs,
143         callbacks=callbacks,
144         verbose=1,
145     )
146
147     actual_epochs = len(history.
148         history["loss"])
149     best_val_auc = max(history.
150         history["val_auc"])
151     print(f"\nTraining complete -
152         epochs run: {actual_epochs} /

```

```

    {epochs} | best_val_auc: {
    best_val_auc:.4f}")
116
117 # Step 4 - Evaluation and
    Visualization
118 metrics - evaluate_model(
119     model=model,
120     test_generator=test_generator
    ,
121     run_name=run_name,
122     output_dir=str(run_dir),
123 )
124
125 # saves {run_name}_acc_loss.png
    into run_dir
126 plot_accuracy_loss(
127     history=history,
128     run_name=run_name,
129     output_dir=str(run_dir),
130 )
131
132 # saves {run_name}_auc_roc.png
    into run_dir
133 plot_auc_roc(
134     model=model,
135     test_generator=test_generator
    ,
136     run_name=run_name,
137     output_dir=str(run_dir),
138 )
139
140 metrics["epochs_run"] -
    actual_epochs
141
142 # Step 5 - Cleanup
143 tf.keras.backend.clear_session()
144
145 print(f"\nCOMPLETED: {run_name}")
146 print("-" * 60)
147
148 return metrics

```

src/evaluate.py

Listing 4: evaluate.py : Model Evaluation

```

1 """
2 evaluate.py
3
4 Evaluates a trained model on the held
    -out test set and persists per-
    run metrics.
5 """
6
7 import os
8 import csv
9 import math
10
11 import numpy as np
12 from sklearn.metrics import (
13     confusion_matrix,
14     precision_score,
15     recall_score,
16     f1_score,
17     roc_auc_score,
18     accuracy_score,
19 )
20
21 from src.visualize import
    plot_confusion_matrix
22
23
24 def evaluate_model(
25     model,
26     test_generator,
27     run_name: str,
28     output_dir: str,
29 ) -> dict:
30     """
31     Evaluate a trained model and
        persist all metrics and plots
        .
32
33     Args:
34         model: Trained tf.
            keras.Model.
35         test_generator: Keras data
            generator for the test
            set.
36
37         Must NOT
            shuffle (
            shuffle-
            False) so
            labels
            stay
            aligned.
38
39         run_name: Unique
            identifier for this run,
            e.g.
            'ResNet50_Adam_LR1e
            -4'.
            Used as
            the
            filename
            stem for
            all saved
            artefacts
            .
40
41         output_dir: Root results/
            directory. Sub-
            directories
            (metrics/,
            confusion_matrices
            /) are
            created
            if absent
            .
42
43     Returns:
44         dict with keys:
45             run_name, architecture,
46             optimizer,
47             learning_rate,
48             accuracy, precision,
49             recall, specificity,
50             f1, auc
51
52     """
53     # Prediction
54     # Reset generator so prediction
55     # starts at the first batch.
56     test_generator.reset()
57     y_prob = model.predict(
58         test_generator, verbose=0).
        ravel()
59     y_pred = (y_prob >= 0.5).astype(
60         int)
61     y_true = test_generator.classes
62
63     # Scalar Metrics
64     accuracy = accuracy_score(y_true,
65                               y_pred)
66     precision = precision_score(
67         y_true, y_pred, zero_division
68         =0)
69     recall = recall_score(y_true,

```

```

        y_pred, zero_division=0)
59     f1 - f1_score(y_true,
        y_pred, zero_division=0)
60     auc - roc_auc_score(y_true,
        y_prob)
61
62     # Specificity: TN / (TN + FP)
63     cm = confusion_matrix(y_true,
        y_pred)
64     tn, fp = cm[0, 0], cm[0, 1]
65     specificity = tn / (tn + fp) if (
        tn + fp) > 0 else 0.0
66
67     # Parse run_name into components
68     # Expected format: {Architecture}
        _{Optimizer}_LR{lr} e.g.
        ResNet50_Adam_LR1e-4
69     parts = run_name.split("_")
70     architecture = parts[0] if len(
        parts) > 0 else run_name
71     optimizer_name = parts[1] if len(
        parts) > 1 else ""
72     learning_rate = parts[2].replace
        ("LR", "") if len(parts) > 2
        else ""
73
74     # Confusion matrix plot
75     cm_dir = os.path.join(output_dir,
        "confusion_matrices")
76     os.makedirs(cm_dir, exist_ok=True
        )
77     plot_confusion_matrix(
78         cm=cm,
79         run_name=run_name,
80         class_names=list(
            test_generator.
            class_indices.keys()),
81         output_dir=cm_dir,
82     )
83
84     # Save metrics CSV (one row per
        run)
85     metrics_dir = os.path.join(
        output_dir, "metrics")
86     os.makedirs(metrics_dir, exist_ok
        =True)
87     csv_path = os.path.join(
        metrics_dir, f"{run_name}.csv
        ")
88
89     metrics = {
90         "run_name": run_name,
91         "architecture": architecture,
92         "optimizer":
            optimizer_name,
93         "learning_rate":
            learning_rate,
94         "accuracy": round(
            accuracy,
            4),
95         "precision": round(
            precision,
            4),
96         "recall": round(recall,
            4),
97         "specificity": round(
            specificity, 4),
98         "f1": round(f1,
            4),
99         "auc": round(auc,
            4),
100     }
101
102     with open(csv_path, "w", newline=
        "") as fh:

```

```

103         writer = csv.DictWriter(fh,
            fieldnames=list(metrics.
            keys()))
104         writer.writeheader()
105         writer.writerow(metrics)
106
107         print(
108             f"[evaluate] {run_name} | "
109             f"acc-{{accuracy:.4f}} prec-{{
                precision:.4f}} rec-{{
                recall:.4f}} "
110             f"spec-{{specificity:.4f}} f1-
                {{f1:.4f}} auc-{{auc:.4f}}")
111     )
112
113     return metrics

```

src/visualize.py

Listing 5: visualize.py : Results Visualization

```

1  """
2  visualize.py
3
4  Visualization utilities for the
        optimizer/learning-rate study.
        Each training run
5  produces three plot types:
6
7  - Accuracy/loss curves -> {run_dir
        }/{run_name}_acc_loss.png
8  - AUC-ROC curves -> {run_dir
        }/{run_name}_auc_roc.png
9  - Confusion matrices -> {run_dir
        }/confusion_matrices/{run_name
        }.png
10 """
11
12 import os
13
14 import numpy as np
15 import matplotlib
16 matplotlib.use("Agg") # non-
        interactive backend - safe for
        headless runs
17 import matplotlib.pyplot as plt
18 import seaborn as sns
19 from sklearn.metrics import roc_curve
        , auc as sklearn_auc
20
21 # Shared style Settings
22
23 _STYLE - "seaborn-v0_8-
        whitegrid"
24 _FIG_DPI - 150
25 _CANCER_COLOR - "#C0392B" # red -
        cancer class
26 _NORMAL_COLOR - "#2980B9" # blue -
        no_cancer class
27
28
29 # Accuracy / Loss curves
30
31 def plot_accuracy_loss(history,
        run_name: str, output_dir: str) ->
        None:
32     """
33     Plot training and validation
        accuracy + loss curves and
        save as PNG.
34

```

```

35     Saves to: {output_dir}/{run_name}
       _acc_loss.png
36
37     Args:
38         history:      tf.keras.
                       callbacks.History object
                       (or its .history dict).
39         run_name:     Stem used for the
                       output filename.
40         output_dir:   Directory in
                       which the figure is saved
                       (typically the run
                       directory).
41     """
42     os.makedirs(output_dir, exist_ok=
43     True)
44     h = history.history if hasattr(
45     history, "history") else
46     history
47     epochs = range(1, len(h["accuracy
48     "] + 1)
49
50     with plt.style.context(_STYLE):
51         fig, (ax_acc, ax_loss) = plt.
52         subplots(1, 2, figsize=
53         (12, 4))
54         fig.suptitle(run_name,
55         fontsize=11, fontweight="
56         bold")
57
58         # Accuracy subplot
59         ax_acc.plot(epochs, h["
60         accuracy"], label="
61         Train Accuracy",
62         linewidth=1.8)
63         ax_acc.plot(epochs, h["
64         val_accuracy"], label="
65         Val Accuracy",
66         linewidth=1.8, linestyle="
67         --")
68         ax_acc.set_title("Accuracy")
69         ax_acc.set_xlabel("Epoch")
70         ax_acc.set_ylabel("Accuracy")
71         ax_acc.legend()
72         ax_acc.set_ylim(0, 1)
73
74         # Loss subplot
75         ax_loss.plot(epochs, h["loss"
76         ], label="Train Loss"
77         , linewidth=1.8)
78         ax_loss.plot(epochs, h["
79         val_loss"], label="Val
80         Loss", linewidth=1.8,
81         linestyle="--")
82         ax_loss.set_title("Loss")
83         ax_loss.set_xlabel("Epoch")
84         ax_loss.set_ylabel("Binary
85         Cross-Entropy")
86         ax_loss.legend()
87
88         fig.tight_layout()
89         out_path = os.path.join(
90         output_dir, f"{run_name}
91         _acc_loss.png")
92         fig.savefig(out_path, dpi=
93         _FIG_DPI, bbox_inches="
94         tight")
95         plt.close(fig)
96
97     print(f"[visualize] Accuracy/Loss
98     curve saved -> {out_path}")
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116

```

```

75
76 # AUC-ROC curve
77
78 def plot_auc_roc(model,
79 test_generator, run_name: str,
80 output_dir: str) -> None:
81     """
82     Compute the ROC curve and save
83     the AUC-ROC plot as PNG.
84
85     Saves to: {output_dir}/{run_name}
86     _auc_roc.png
87
88     Args:
89         model:          Trained tf.
90         keras.Model.
91         test_generator: Test
92         generator with shuffle=
93         False.
94         run_name:       Stem used for
95         the output filename.
96         output_dir:     Directory in
97         which the figure is saved
98         (typically the run
99         directory).
100     """
101     os.makedirs(output_dir, exist_ok=
102     True)
103
104     test_generator.reset()
105     y_prob = model.predict(
106     test_generator, verbose=0).
107     ravel()
108     y_true = test_generator.classes
109
110     fpr, tpr, _ = roc_curve(y_true,
111     y_prob)
112     roc_auc = sklearn_auc(fpr,
113     tpr)
114
115     with plt.style.context(_STYLE):
116         fig, ax = plt.subplots(
117         figsize=(6, 5))
118         ax.plot(fpr, tpr, color=
119         _CANCER_COLOR, linewidth=
120         2,
121         label=f"AUC - {
122         roc_auc:.4f}")
123         ax.plot([0, 1], [0, 1], color=
124         "grey", linestyle="--",
125         linewidth=1,
126         label="Random
127         Classifier")
128         ax.set_xlabel("False Positive
129         Rate")
130         ax.set_ylabel("True Positive
131         Rate")
132         ax.set_title(f"ROC Curve - {
133         run_name}", fontweight="
134         bold")
135         ax.legend(loc="lower right")
136         ax.set_xlim(0, 1)
137         ax.set_ylim(0, 1.02)
138
139         fig.tight_layout()
140         out_path = os.path.join(
141         output_dir, f"{run_name}
142         _auc_roc.png")
143         fig.savefig(out_path, dpi=
144         _FIG_DPI, bbox_inches="
145         tight")
146         plt.close(fig)

```

```

117     print(f"[visualize] AUC-ROC curve
        saved -> {out_path}")
118
119
120 # Confusion matrix
121
122 def plot_confusion_matrix(
123     cm: np.ndarray,
124     run_name: str,
125     class_names: list,
126     output_dir: str,
127 ) -> None:
128     """
129     Plot a seaborn heatmap of the
        confusion matrix and save as
        PNG.
130
131     Saves to: {output_dir}/{run_name}
        .png (output_dir is
        typically confusion_matrices/
        under the run directory)
132
133     Args:
134         cm:          2x2 numpy
                        confusion matrix from
                        sklearn.
135         run_name:     Stem used for
                        the output filename.
136         class_names:  List of class
                        label strings, e.g. ['
                        cancer', 'no_cancer'].
137         output_dir:   Directory in
                        which the figure is saved
                        (typically {run_dir}/
                        confusion_matrices/).
138     """
139     os.makedirs(output_dir, exist_ok=
        True)
140
141     with plt.style.context(_STYLE):
142         fig, ax = plt.subplots(
            figsize=(5, 4))
143
144         # Annotate each cell with
            count and row percentage
145         total_per_row = cm.sum(axis=
            1, keepdims=True)
146         pct = cm / total_per_row.clip
            (min=1) * 100
147         annot = np.array(
148             [[f"{cm[i, j]}\n({pct[i,
                j]:.1f}%)" for j in
                range(cm.shape[1])]
149              for i in range(cm.shape
                [0])]
150         )
151
152         sns.heatmap(
153             cm,
154             annot=annot,
155             fmt="",
156             cmap="Blues",
157             xticklabels=class_names,
158             yticklabels=class_names,
159             linewidths=0.5,
160             ax=ax,
161             cbar_kws={"shrink":
                0.75},
162         )
163         ax.set_xlabel("Predicted
            Label", fontsize=10)
164         ax.set_ylabel("True Label",
            fontsize=10)

```

```

165         ax.set_title(f"Confusion
            Matrix - {run_name}",
            fontweight="bold",
            fontsize=11)
166
167         fig.tight_layout()
168         out_path = os.path.join(
            output_dir, f"{run_name}.
            png")
169         fig.savefig(out_path, dpi=
            _FIG_DPI, bbox_inches="
            tight")
170         plt.close(fig)
171
172     print(f"[visualize] Confusion
        matrix saved -> {out_path}")

```

src/run_all.py

Listing 6: run_all.py : Experiment Orchestrator

```

1  """
2  run_all.py
3
4  Master orchestration script for all
    72 simulation runs.
5  Builds the shared data pipeline once,
    then loops over every
    combination of:
6      - 4 architectures : VGG19,
        EfficientNetB3, ResNet50,
        DenseNet121
7      - 6 optimizers : Adam, Adagrad,
        Adamax, AdaDelta, SGD, RMSProp
8      - 3 learning rates: 1e-4, 1e-5, 1e-
        6
9
10 Run directly: python src/run_all.py
11 """
12
13 import itertools
14 import pathlib
15 import random
16
17 import numpy as np
18 import pandas as pd
19 import tensorflow as tf
20 from tensorflow.keras.preprocessing.
    image import ImageDataGenerator
21
22 from src.model_builder import
    build_model
23 from src.train import train_model
24
25
26 # Configuration
27
28 RANDOM_SEED      - 42
29 EPOCHS           - 50
30 BATCH_SIZE       - 32
31 INPUT_SIZE       - (256, 256)
32
33 ARCHITECTURES    - ['VGG19', '
    EfficientNetB3', 'ResNet50', '
    DenseNet121']
34 OPTIMIZERS       - ['Adam', 'Adagrad',
    'Adamax', 'AdaDelta', 'SGD', '
    RMSProp']
35 LEARNING_RATES  - [1e-4, 1e-5, 1e-6]
36
37

```

```

38 # Helpers
39
40 def _seed_everything(seed: int) ->
    None:
41     random.seed(seed)
42     np.random.seed(seed)
43     tf.random.set_seed(seed)
44
45
46 # Main
47
48 def run_all() -> None:
49     _seed_everything(RANDOM_SEED)
50
51     project_root = pathlib.Path(
52         __file__).resolve().parent.
53         parent
54     processed_dir = project_root / "
55         dataset" / "processed"
56     results_dir = project_root / "
57         results"
58     summary_dir = results_dir / "
59         summary"
60     summary_dir.mkdir(parents=True, e
61         xist_ok=True)
62
63     # Section 2 - Shared Data
64     Pipeline (constructed ONCE,
65         reused across all 72)
66
67     image_generatorFullAugment =
68         ImageDataGenerator(
69             samplewise_center=True,
70             samplewise_std_normalization=
71                 True,
72             rotation_range=30,
73             width_shift_range=0.15,
74             height_shift_range=0.15,
75             shear_range=0.3,
76             zoom_range=0.30,
77             horizontal_flip=True,
78             fill_mode="nearest",
79         )
80
81     image_generatorNoAugment =
82         ImageDataGenerator(
83             samplewise_center=True,
84             samplewise_std_normalization=
85                 True,
86         )
87
88     train_generator =
89         image_generatorFullAugment.
90         flow_from_directory(
91             str(processed_dir / "train"),
92             target_size=INPUT_SIZE,
93             batch_size=BATCH_SIZE,
94             class_mode="binary",
95             shuffle=True,
96             seed=RANDOM_SEED,
97         )
98
99     val_generator =
100         image_generatorFullAugment.
101         flow_from_directory(
102             str(processed_dir / "val"),
103             target_size=INPUT_SIZE,
104             batch_size=1,
105             class_mode="binary",
106             shuffle=True,
107             seed=RANDOM_SEED,
108         )
109
110     test_generator =
111         image_generatorNoAugment.
112         flow_from_directory(
113             str(processed_dir / "test"),
114             target_size=INPUT_SIZE,
115             batch_size=1,
116             class_mode="binary",
117             shuffle=False,
118         )
119
120     print(f"Train : {train_generator.
121         samples} images")
122     print(f"Val : {val_generator.
123         samples} images")
124     print(f"Test : {test_generator.
125         samples} images")
126     print(f"Classes: {train_generator.
127         class_indices}")
128     print()
129     print(
130         "[WARNING] EXPERIMENTAL
131         INTEGRITY NOTE:
132         generators are
133         constructed ONCE here "
134         "and shared across all 72
135         runs. Do not re-
136         initialize them inside
137         the loop - "
138         "doing so would reshuffle the
139         training data
140         differently per run and
141         confound "
142         "optimizer/LR comparisons."
143     )
144
145     # Section 3 - 72-Run Training
146     Loop
147
148     experiment_matrix = list(
149         itertools.product(
150             ARCHITECTURES, OPTIMIZERS,
151             LEARNING_RATES))
152     total_runs = len(ex
153         periment_matrix)
154
155     print(f"\nTOTAL RUNS: {total_runs
156         }")
157     print("Starting simulation.\n")
158
159     all_metrics: list[dict] = []
160
161     for counter, (arch, opt, lr) in
162         enumerate(experiment_matrix,
163             start=1):
164         lr_str = f"{lr:.0e}"
165         print(f"\n{'-' * 60}")
166         print(f"Run {counter:02d}/{
167             total_runs} | {arch} |
168             {opt} | LR-{lr_str}")
169         print(f"{'-' * 60}")
170
171         model = build_model(
172             architecture_name=arch,
173             optimizer_name=opt,
174             learning_rate=lr,
175         )
176
177         metrics = train_model(
178             model=model,
179             train_generator=
180                 train_generator,
181             val_generator=
182                 val_generator,

```

```

140         test_generator-
141             test_generator,
142         architecture=arch,
143         optimizer_name=opt,
144         learning_rate=lr,
145         results_base_dir-
146             results_dir,
147         epochs=EPOCHS,
148     )
149     all_metrics.append(metrics)
150     train_generator.reset()
151     test_generator.reset()
152
153     # Section 4 - Aggregate Results
154
155     master_df = pd.DataFrame(
156         all_metrics)
157     summary_csv = summary_dir / "
158         master_results.csv"
159     master_df.to_csv(summary_csv,
160         index=False)
161     print(f"\nMaster results saved ->
162         {summary_csv}")
163     print(master_df.to_string())
164     print(f"\nALL {total_runs}
165         SIMULATIONS COMPLETE")
166
167     # Entry point
168
169     if __name__ == "__main__":
170         run_all()

```
